

Confidential Transfers for Multi-Purpose Tokens on the XRP Ledger

Murat Cenk
Ripple Research
mcenk@ripple.com

Aanchal Malhotra
Ripple Research
amalhotra@ripple.com

Joseph A. Akinyele
Ripple Research
jakinyele@ripple.com

Abstract

We introduce Confidential Transfers for Multi-Purpose Tokens (Confidential MPTs) on the XRP Ledger, a cryptographic extension of the XLS-33 token standard that enables confidential balances and hidden transfer amounts while preserving public supply verifiability. The protocol replaces plaintext per-account balances with EC-ElGamal ciphertexts and employs non-interactive zero-knowledge proofs to enforce transfer correctness, balance sufficiency, and the invariant $\text{OutstandingAmount} \leq \text{MaxAmount}$ without requiring decryption by validators. Confidentiality is scoped to transaction amounts and account balances; sender and receiver identities remain public, preserving XRPL’s account-based execution model. Our design maintains full compatibility with existing MPT semantics: public and confidential balances coexist, issuance rules remain unchanged, and the issuer’s designated second account is treated identically to other holders. The protocol further supports issuer-controlled operations, including freeze and clawback, without weakening supply soundness. To accommodate regulatory and institutional requirements, Confidential MPTs provide cryptographic auditability through an on-chain selective-disclosure model based on multi-ciphertext balance representations and equality proofs, while remaining compatible with simpler issuer-mediated audit models. We present a complete protocol specification, a security analysis under standard discrete-logarithm assumptions, and an open-source reference implementation (mpt-crypto) that realizes the required cryptographic primitives. Experimental evaluation demonstrates that confidential transfers can be verified within XRPL validator performance constraints, with proof sizes and verification costs suitable for production deployment.

Key words: Confidential Transactions; Zero-Knowledge Proofs; Homomorphic Encryption; Bulletproofs; Supply Verifiability; Auditability; Blockchain Protocols; XRP Ledger

1 Introduction

Public blockchains provide exceptional transparency and verifiability, but this strength becomes a liability in many institutional and privacy-sensitive settings. On ledgers such as the XRP Ledger (XRPL), all token balances and transfer amounts are visible to anyone who can read the chain. While this transparency simplifies auditing and supply enforcement, it also exposes positions, flows, and counterparty relationships that many real-world actors consider commercially or regulatorily sensitive.

The Multi-Purpose Token (MPT) standard (XLS-33) [XRP24] extends XRPL with a flexible, issuer-controlled token framework that supports a wide range of assets and business logic. However, XLS-33 inherits the same transparency model: all MPT balances and transfers are fully visible in plaintext. This visibility is a fundamental blocker for several classes of applications: (i) institutional tokenization where portfolio sizes and flows should not be public, (ii) enterprise settlement workflows where counterparties require confidentiality, and (iii) regulated environments where auditors need selective visibility, but the general public should not see per-account positions. In parallel, recent advances in zero-knowledge proofs (ZKPs) and efficient elliptic-curve cryptography have made it possible to hide transaction amounts while still enforcing strong global invariants such as conservation of value and issuance caps.

This work brings these ideas to the XRPL MPT model by introducing Confidential Multi-Purpose Tokens (Confidential MPTs): tokens whose balances and transfer amounts are kept encrypted on-ledger, yet whose total supply and issuance constraints remain publicly verifiable. At a high level, our objective is to extend the XLS-33 MPT standard with support for confidential balances and transfers, while preserving several core properties that are

fundamental to the XRPL’s token model. First, the system must maintain *supply soundness*: the total outstanding supply of a token must never exceed the issuer-defined maximum, as enforced in XLS-33 by the invariant $\text{OutstandingAmount} \leq \text{MaxAmount}$. Second, this supply invariant must remain *publicly verifiable*. Any validator or external observer should be able to independently verify issuance limits and circulation behavior directly from the public ledger state, without requiring access to private keys or off-ledger information. Third, the design must preserve operational compatibility with existing XLS-33 semantics, so that issuers and holders can continue to rely on established accounting models, reserve management practices, and integration patterns without migrating to an entirely new token abstraction.

A naive approach that simply encrypts all on-ledger balances fails to satisfy these requirements. If per-account balances are replaced with ciphertexts, validators can no longer compute the outstanding supply, and external observers lose the ability to independently check supply bounds or reconcile issuance with circulation. Moreover, practical institutional use cases demand more than merely obscuring numerical values. Real-world deployments typically require controlled forms of compliance visibility, such as granting auditors read-only access to balances without conferring spending authority, as well as issuer-enforced mechanisms for freezing or clawing back funds under regulatory or contractual obligations.

This leads to the central problem addressed by this work: how can XRPL support confidential MPT balances and transfers in a way that keeps individual amounts and per-account balances hidden, while simultaneously (i) preserving the existing XLS-33 supply semantics and invariants, and (ii) enabling flexible, controllable audit and compliance views that meet institutional requirements?

1.1 Design Goals and Challenges

Designing Confidential MPTs requires simultaneously satisfying confidentiality, correctness, and compatibility constraints that are often in tension with one another. A central requirement is the preservation of XLS-33 semantics. In XLS-33, the `OutstandingAmount` (OA) is defined as the sum of all non-issuer balances, and issuance is constrained by the invariant $\text{OA} \leq \text{MaxAmount}$ (MA). Any confidential extension must preserve this definition exactly, so that existing infrastructure, tooling, and issuer accounting models remain valid. Redefining OA to account for hidden supply would complicate protocol analysis, weaken public verifiability, and introduce friction for downstream integrations that rely on established XLS-33 behavior. At the same time, Confidential MPTs must ensure the confidentiality of per-account balances and transfer amounts. Validators and external observers should not learn individual balances or the amounts moved in confidential transfers, yet validators must still be able to verify transaction correctness. This necessitates an encryption scheme with additive homomorphism, together with ZKPs that establish balance sufficiency and value consistency without revealing the underlying plaintext amounts. Despite hiding individual balances, the system must retain public, trustless auditability of total supply. Observers should be able to verify that aggregate issuance never exceeds the issuer-defined maximum, without access to any private decryption keys. To achieve this, the protocol introduces an additional plaintext ledger field, `ConfidentialOutstandingAmount` (COA), which tracks the total confidential portion of circulation held by non-issuer accounts, including the issuer’s designated second account. The ledger maintains the relationship

$$\text{OA} = \text{PublicOutstanding} + \text{COA}, \quad \text{OA} \leq \text{MA},$$

ensuring that supply limits remain publicly enforceable even though the distribution of confidential balances across individual holders is hidden.

Practical deployments also require flexible mechanisms for compliance and selective disclosure. In institutional settings, this may include trust-minimized, on-chain auditing by designated regulators or custodians, as well as simpler issuer-mediated auditing in which view keys are shared directly with authorized parties. Supporting these models requires representing the same confidential balance as multiple ciphertexts encrypted under different public keys (e.g., holder, issuer, and optional auditor keys), together with zero-knowledge equality proofs that cryptographically bind all ciphertexts to the same underlying amount. Confidential transfers further introduce usability challenges related to proof staleness and replay resistance. A user may generate a ZKP with respect to their current confidential balance state. If the account receives additional confidential transfers before the transaction is validated, the underlying balance state changes, and the proof—constructed under the earlier assumptions—no longer corresponds to the ledger state at validation time. This phenomenon is referred to as the stale proof problem. The protocol must therefore provide a ledger-level structure that keeps the balance referenced by proofs stable,

cleanly handles incoming transfers, and prevents replay of outdated proofs, without forcing wallets to constantly regenerate proofs in response to unrelated inbound activity. Finally, performance and on-ledger verification cost are critical considerations. Naive bit-decomposition range proofs scale linearly with the bit-length of the amount and quickly become impractical due to excessive transaction size and verification overhead. Confidential MPTs therefore require compact range proofs, such as Bulletproofs, whose proof size grows logarithmically in the range and whose verification cost is acceptable for XRPL validators operating at scale.

1.2 Our Approach and Contributions

We propose Confidential MPTs as an extension to XLS-33 that enables confidential balances and transfer amounts while preserving XRPL’s existing supply semantics and validation model. The design combines ledger-level accounting mechanisms with cryptographic enforcement to ensure that confidentiality, correctness, and auditability coexist without redefining the core Multi-Purpose Token abstraction.

This protocol design uses issuer second-account model that enables confidential circulation without modifying XLS-33 issuance semantics. The issuer designates a second, issuer-controlled account that is treated by the ledger as an ordinary, non-issuer holder. Confidential circulation is initiated through a public conversion transaction executed by a holder, which moves funds from that holder’s public balance into its confidential balance. The issuer’s second account participates identically to any other holder account and does not possess any special conversion privilege. Because the second account is a non-issuer, its balance remains included in the outstanding amount (OA) exactly as defined in XLS-33. Consequently, validators continue to enforce the invariant $OA \leq MA$ using only plaintext ledger fields, while confidential transfers remain pure redistributions that preserve both OA and confidential aggregate supply. This model is realized through a dedicated transaction family consisting of convert, confidential send, merge inbox, convert back, and confidential claw back operations, each designed to preserve XRPL’s native issuance and compliance semantics under confidential balance handling.

To address proof staleness and replay risks in XRPL’s account-based execution model, we integrate a split-balance confidential state structure with explicit version binding into the issued-asset ledger model. Each account maintains two encrypted balances: a confidential spending balance CB_S , which serves as the reference state for outgoing transfer proofs, and a confidential inbox balance CB_{IN} , which accumulates incoming transfers. A dedicated merge inbox transaction homomorphically consolidates CB_{IN} into CB_S and resets the inbox to a canonical encryption of zero. Any transaction that modifies CB_S including `CONFIDENTIALMPTSEND`, `CONFIDENTIALMPTMERGEINBOX`, `CONFIDENTIALMPTCONVERTBACK`, and `CONFIDENTIALMPTCLAWBACK` increments a monotonically increasing version counter, and all ZKPs are bound to its current value. This prevents replay of stale proofs while avoiding proof invalidation under unrelated incoming transfers.

Confidential MPTs further adapt a multi-ciphertext balance architecture to reconcile confidentiality with issuer integrity tracking and optional auditor visibility. Each confidential balance is represented by EC–ElGamal ciphertexts encrypted under multiple public keys: the holder’s key for spending control, the issuer’s key for integrity tracking, and optionally an auditor’s key for selective disclosure. Zero-knowledge equality proofs enforce consistency across ciphertext representations. Range and balance-sufficiency proofs, instantiated using Bulletproofs, guarantee that transferred amounts are non-negative and do not exceed the sender’s available balance without revealing values. At the issuance layer, a plaintext field `COA` tracks the aggregate confidential supply and is updated only by transactions that explicitly reveal amounts, such as conversion and clawback operations, preserving public verification of supply limits.

Finally, we provide an open-source reference implementation (`mpt-crypto`) of the required cryptographic primitives and proof constructions, together with performance measurements showing that confidential transfers can be verified within XRPL validator execution constraints.

1.3 Organization

The remainder of this document is organized as follows. Section 2 reviews prior work on confidential transactions. Section 3 presents the cryptographic and ledger-level background required to understand Confidential Multi-Purpose Tokens. Section 4 defines the system model, threat assumptions, and the confidentiality and auditability goals of the protocol. Section 5 provides an overview of the protocol architecture, including ledger extensions and account roles.

Section 6 presents the Confidential MPT protocol, covering transaction semantics, state transitions, and associated ZKPs. Section 7 describes auditability and compliance mechanisms, including issuer visibility and auditor access models. Section 8 analyzes the security properties of the protocol, including supply soundness, balance consistency, replay protection, and privacy guarantees. Section 9 evaluates performance characteristics, including transaction size and verification costs. Section 10 discusses limitations of the current design and possible directions for future work. Finally, Section 11 concludes the paper.

2 Related Work

Confidential transactions on blockchains aim to conceal transaction values and, in some systems, participant identities, while preserving public verifiability and ledger consistency. Existing approaches can be broadly grouped according to the ledger model they target: UTXO-based systems and account-based systems.

UTXO-based confidential payment systems, exemplified by Monero and Zcash, represent value as consumable outputs created and spent by transactions. Monero achieves confidentiality through Ring Signatures, Ring Confidential Transactions (RingCT), and stealth addresses, providing sender ambiguity and hidden transaction amounts [NM16]. Zcash adopts zk-SNARK-based shielded transactions that hide both transferred values and participant addresses while preserving global consistency through ZKP verification [BS⁺14]. These systems benefit from the natural locality of UTXO ownership, where each output can be independently consumed and recreated.

In contrast, account-based ledgers maintain persistent balances associated with accounts, making confidentiality more challenging because balance updates are stateful and directly linked across transactions. Zether [BAMB20] introduced one of the first practical confidential payment systems for account-based ledgers by combining EC-ElGamal encryption with ZKPs to enable confidential transfers while preserving balance correctness. Similar design principles later appeared in confidential asset proposals for Solana [Lab22], Aptos [Lab23], and auditable account-based payment systems such as PGC [CMTA20]. These systems generally rely on additive homomorphic encryption to support ciphertext-domain balance updates. Anonymous extensions of Zether further improve sender anonymity through many-out-of-many proofs, though at significantly higher proof complexity and transaction size [Dia21]. Solana adopts twisted ElGamal variants to improve proof efficiency while preserving additive homomorphism [Sol24, Lab22], whereas recent FHE-based account systems explore stronger confidentiality at substantially higher computational cost [MS25].

Among homomorphic encryption approaches, EC-ElGamal remains the dominant design choice due to its efficiency and compatibility with elliptic-curve-based ledger infrastructures. In such systems, plaintext amounts are encoded as elliptic curve points mG , enabling direct homomorphic addition of encrypted balances. A practical limitation is that decryption reveals only the group element mG , requiring recovery of m through a bounded discrete logarithm computation over a small plaintext range. Efficient small-range elliptic-curve discrete logarithm algorithms therefore become a practical requirement for confidential balance systems. Several methods have been proposed for this purpose. Shanks' Baby-Step Giant-Step (BSGS) algorithm [GWZ17] reduces decryption complexity to square-root time at the cost of substantial memory usage. Bernstein and Lange [BL12, BL13] improve practical BSGS performance through arithmetic and memory optimizations, while Pollard's Kangaroo algorithm [Pol00] provides a memory-efficient alternative with comparable expected runtime. More recent blockchain-oriented optimizations include compressed discrete-log lookup tables by Chatzigiannis et al. [CCN21] and FastECDLP by Tang et al. [TLC⁺23], which demonstrates sub-millisecond recovery for practical plaintext ranges relevant to encrypted balance systems.

Zether further proposes practical client-side techniques to mitigate decryption cost. All discrete-log recovery is performed off-chain, preventing decryption overhead from affecting consensus. The design additionally exploits balance locality by assuming that users maintain a good estimate of their current balance, allowing discrete-log search to be restricted to a narrow interval around the expected value. For larger plaintext domains, Zether applies a chunking strategy that splits balances into smaller limbs encrypted independently, reducing decryption complexity while preserving correctness via additional equality proofs.

Our design follows the account-based confidential transaction line but targets the XRP Ledger's issued-asset architecture, where confidential transfers must coexist with issuer-controlled supply accounting, trust relationships, and native ledger semantics. Unlike general-purpose smart-contract systems, this requires protocol-level integration that preserves XRPL transaction and ledger consistency while supporting efficient encrypted balance updates.

3 Background

XLS-33 defines a token issuance through an **MPTokenIssuance** ledger object containing the issuer identifier, token identifier, maximum supply **MaxAmount**, and **OutstandingAmount** (OA), which tracks the total supply held by all non-issuer accounts. Each holder maintains a corresponding **MPToken** object storing its plaintext balance. Validators enforce the invariant

$$OA \leq \text{MaxAmount},$$

with

$$OA = \sum_{\text{non-issuer } A} \text{Balance}(A),$$

so supply verification remains fully transparent and derives directly from public ledger state.

Confidential MPTs are instantiated over the secp256k1 elliptic-curve group and combine EC-ElGamal encryption, Pedersen commitments, and non-interactive zero-knowledge proofs. EC-ElGamal provides additive homomorphism for confidential balance updates under recipient-specific public keys, while Pedersen commitments support range proofs without revealing amounts. In particular, **CONFIDENTIALMPTSEND** proves both the transfer amount and the post-transfer remainder using aggregated Bulletproofs over 64-bit values. Additional Schnorr-style proofs ensure key possession, ciphertext consistency, plaintext equality across ciphertext domains, and linkage between ElGamal ciphertexts and Pedersen commitments. All proofs are Fiat–Shamir transformed and bound to a transaction-specific context identifier. Detailed constructions are deferred to Appendix A.

4 System Model and Design Goals

Confidential MPTs extend the XLS-33 MPT [XRP24] framework by introducing transaction-level confidentiality for balances and transfer amounts without altering existing supply semantics. The system involves four participant classes: the issuer, holders, validators, and optionally auditors. The issuer creates an MPT issuance under XLS-33 and may operate a designated second account treated as an ordinary holder. Holders are standard XRPL accounts that may simultaneously maintain public and confidential balances of the same token. Validators execute deterministic ledger rules, verify cryptographic proofs, and apply state transitions without learning confidential transfer amounts or per-account confidential balances. Auditors, when enabled, receive balances encrypted under their own public key, allowing independent balance verification without interaction with the issuer or holder.

Each issuance is represented by an **MPTokenIssuance** object containing the standard XLS-33 fields together with confidential-transfer configuration, including an issuer ElGamal public key, optional auditor settings, and a public **COA** (ConfidentialOutstandingAmount) field tracking aggregate confidential supply. Since confidential balances are stored as ciphertexts, validators cannot derive total supply by summing per-account balances. Supply consistency is therefore maintained through the invariant $OA = \text{PublicOutstanding} + \text{COA}$, where **OA** denotes **OutstandingAmount**, **PublicOutstanding** is derived from visible balances, and **COA** represents aggregate confidential holdings. The **COA** field changes only during conversion-style transactions that explicitly reveal an amount, while confidential transfers between holders leave both **OA** and **COA** unchanged. For each holder, confidential state is stored in an **MPToken** object extending XLS-33 with an encrypted confidential spending balance **CB_S**, an encrypted inbox balance **CB_{IN}**, a version counter **CB_S.version**, the holder’s ElGamal public key, and issuer- and optionally auditor-encrypted mirror ciphertexts.

We assume a probabilistic polynomial-time adversary that may control arbitrary network participants except validators, which are modeled as honest-but-curious. The adversary observes the full public ledger state, including ciphertexts, public keys, transaction metadata, and zero-knowledge proofs, and may submit arbitrary transactions consistent with XRPL authorization rules. It may attempt malformed ciphertext construction, invalid proof generation, replay, or balance underflow attacks, but cannot break discrete-logarithm hardness assumptions or violate Bulletproof soundness. Private ElGamal keys remain secret unless explicitly disclosed. The issuer may be malicious, but cannot exceed **MaxAmount** or create confidential balances without producing valid proofs.

The protocol aims to provide confidentiality of transfer amounts and confidential balances while preserving public supply verifiability. Transactions such as **ConfidentialMPTSend** hide transferred values from validators and observers, who see only ciphertexts and associated proofs. Correctness is enforced through equality proofs across

ciphertext domains, range proofs preventing overspending, and proof binding to transaction fields and the current $\text{CB}_S.\text{version}$ for replay protection. Ledger updates remain fully deterministic, including canonical inbox resets.

Confidential MPTs intentionally do not provide sender or receiver anonymity: account identities remain public, consistent with XRPL’s account-based execution model. Conversion-style transactions explicitly reveal amounts to preserve XLS-33 supply semantics, and confidential integration with other XRPL features such as the DEX or escrow remains outside the current scope.

The design also reflects XRPL execution constraints: proof verification must remain efficient, transaction sizes compatible with ledger processing, and no trusted setup may be introduced. For this reason, Bulletproofs are used instead of SNARK-based constructions, and ledger extensions are limited to the minimum additional fields and transaction types required for confidentiality.

5 Confidential MPT Architecture

Confidential MPTs rely on three architectural mechanisms that jointly preserve XLS-33 supply semantics while enabling confidential balances and transfers: the issuer second-account model, the split-balance model, and a multi-ciphertext balance representation.

To preserve the meaning of `OutstandingAmount` (OA), confidential supply is introduced through a designated issuer-controlled second account treated as an ordinary holder by the ledger. Since validators cannot derive total confidential circulation from encrypted balances, conversion-style transactions explicitly reveal an amount m and update both OA and COA, where COA tracks aggregate confidential supply. Confidential transfers between holders then redistribute existing confidential balances without modifying either quantity, preserving the invariant

$$\text{OA} = \text{PublicOutstanding} + \text{COA}.$$

This keeps XLS-33 supply validation unchanged: only conversion-style transactions modify confidential circulation. To ensure proof stability, each holder’s confidential balance is split into a confidential spending balance CB_S and an inbox balance CB_{IN} . Outgoing zero-knowledge proofs reference only CB_S , while incoming transfers accumulate in CB_{IN} . Newly received funds become spendable only after an explicit merge:

$$\text{CB}_S \leftarrow \text{CB}_S \oplus \text{CB}_{\text{IN}}, \quad \text{CB}_{\text{IN}} \leftarrow \text{Enc}(0), \quad \text{CB}_S.\text{version} \leftarrow \text{CB}_S.\text{version} + 1.$$

This avoids proof invalidation caused by concurrent incoming transfers and provides replay protection through version binding. Moreover, each confidential balance is represented by multiple ElGamal ciphertexts encrypting the same amount under different public keys:

$$(\text{Enc}_{\text{pk}_H}(m), \text{Enc}_{\text{pk}_I}(m), \text{Enc}_{\text{pk}_A}(m)),$$

corresponding to the holder, issuer, and optional auditor. Equality proofs ensure consistency across all ciphertext domains. The holder ciphertext enables spending, the issuer ciphertext supports confidential circulation tracking and administrative functions such as clawback, and optional auditor ciphertexts provide selective disclosure without altering ledger semantics. Ledger updates remain fully deterministic by exploiting ElGamal homomorphism:

$$\text{Enc}(m_1) \oplus \text{Enc}(m_2) = \text{Enc}(m_1 + m_2).$$

Inbox resets use a canonical deterministic encryption of zero derived from ledger identifiers, ensuring validators never rely on prover-supplied randomness. Together, these mechanisms guarantee supply consistency, ciphertext coherence, and deterministic ledger execution, forming the basis of the transaction protocol described next.

6 Confidential MPT Protocol

This section presents the transaction-level protocol that realizes confidential balances and transfers on the XRPL. Each transaction defines a deterministic state transition on `MPTokenIssuance` and `MPToken` objects, verified by ZKPs where required, following the architectural principles of Section 5. For a holder account A and token issuance (I, C) ,

the ledger maintains a per-holder **MPToken** object containing both public and confidential state. In the confidential extension, this includes a confidential spending balance $\text{CB}_S(A)$, a confidential inbox balance $\text{CB}_{\text{IN}}(A)$, and a monotonically increasing version counter $\text{CB}_S.\text{version}(A)$ used for replay protection. Parallel issuer- and optionally auditor-encrypted ciphertexts, denoted $\text{Enc}_I(A)$ and $\text{Enc}_A(A)$, maintain mirror views of the holder’s confidential balance, while the holder’s ElGamal public key pk_A is stored for ciphertext validation and proof verification.

At the issuance level, **MPTokenIssuance** stores the public supply parameters OA , COA , and MA , together with the issuer public key pk_I and optional auditor configuration. Confidential balance updates exploit EC-ElGamal homomorphism,

$$\text{Enc}(m_1) \oplus \text{Enc}(m_2) = \text{Enc}(m_1 + m_2),$$

allowing validators to update ciphertext balances without decryption. Inbox resets use a canonical deterministic encryption of zero,

$$\text{EncZero}(A, I, C) = \text{Enc}_{\text{pk}_A}(0; r_{\text{det}}),$$

where r_{det} is derived from domain-separated public ledger inputs so that all validators compute the same ciphertext.

All Confidential MPT transactions follow standard XRPL authorization rules: transactions are signed using the account’s XRPL signing key, while ElGamal keys are used exclusively for confidential balance encryption and ZKPs.

6.1 ConfidentialMPTConvert: Public to Confidential and Account Initialization

The **ConfidentialMPTConvert** transaction converts a publicly visible MPT balance into its confidential form. It has a single, uniform meaning for all holders, including the issuer’s designated second account. The issuer does not directly mint confidential supply. Instead, it must first transfer public tokens to its second account using standard XLS-33 mechanisms; that account—treated as an ordinary non-issuer holder—then performs **ConfidentialMPTConvert**. This preserves XLS-33 issuance semantics and avoids introducing special-case confidential minting logic. Semantically, **ConfidentialMPTConvert** reveals an amount m and moves it from the holder’s public balance into an encrypted confidential balance under the holder’s ElGamal public key. The conversion increases COA by m while preserving the invariant that OA remains unchanged.

Account initialization and key registration. Participation in the Confidential MPT protocol requires on-ledger registration of an ElGamal public key pk_A . This initialization is performed via **ConfidentialMPTConvert** itself and does not require a separate enrollment transaction. Before an account A can receive, hold, or spend confidential value, it must publish pk_A . Validators reject any confidential transaction targeting an account whose ElGamal public key is not already registered. Initialization may be performed in two equivalent ways:

1. **Zero-amount initialization.** The holder submits a **ConfidentialMPTConvert** transaction with $\text{Amount} = 0$. This registers pk_A and initializes the confidential balance fields without moving value or affecting supply fields.
2. **Non-zero conversion.** The holder submits a **ConfidentialMPTConvert** transaction with $\text{Amount} > 0$, simultaneously initializing the confidential state and converting public balance into confidential form.

Once the validator processes this transaction, the holder’s **MPToken** object is updated to include the provided ElGamal public key pk_A , and the confidential spending balance version $\text{CB}_S.\text{Version}$ is initialized. This makes the account cryptographically addressable: other participants can now encrypt confidential amounts under pk_A , and any subsequent ZKPs involving the holder’s spending balance are bound to a well-defined initial version. The **MPToken** object contains the issuer-encrypted mirror field, which tracks aggregate confidential circulation and serves as the authoritative reference for audit and clawback operations. No additional per-issuer initialization step is required. When the issuer’s designated second account executes **ConfidentialMPTConvert**, it undergoes the same holder-side initialization semantics as any other account.

State transition. For a holder account A (which may be the issuer’s second account), the ledger applies the following deterministic state updates:

$$\begin{aligned} \text{CB}_{\text{IN}}(A) &\leftarrow \text{CB}_{\text{IN}}(A) \oplus \text{Enc}_A(m), \\ \text{Enc}_I(A) &\leftarrow \text{Enc}_I(A) \oplus \text{Enc}_I(m), \\ \text{COA} &\leftarrow \text{COA} + m. \end{aligned}$$

The confidential amount is credited to the inbox balance to prevent immediate proof staleness. A parallel issuer-encrypted mirror is updated to maintain audit and clawback consistency. If an auditor policy is active for the issuance, an additional auditor-encrypted ciphertext may be updated analogously. Because the converting account is already counted as a non-issuer holder, OA remains unchanged.

Encryption Verification. Each `ConfidentialMPTConvert` transaction includes ElGamal ciphertexts encrypting a publicly revealed amount m . Since the plaintext value is known, the protocol does not require ZKPs for encryption correctness. Instead, the prover reveals the encryption randomness r , allowing validators to deterministically verify that each ciphertext correctly encrypts the public amount. Concretely, for each ciphertext included in the transaction (holder, issuer, and optional auditor), validators check that: $C_1 = rG$ and $C_{2,i} = mG + r \cdot \text{pk}_i$, where pk_i is the corresponding recipient public key. These checks ensure that all confidential balances created by the conversion are mutually consistent and correspond exactly to the revealed amount m .

Validation Rules. Before applying the state transition, validators perform the following checks:

- confidential transfers are enabled for the token issuance;
- the revealed amount m is non-negative and conforms to XRPL decimal semantics;
- the converting account has sufficient public balance to cover the conversion;
- all submitted ciphertexts are well-formed secp256k1 group elements;
- the revealed encryption randomness r correctly reconstructs each ciphertext for the public amount m under the corresponding public keys;
- if a confidential public key is registered in this transaction, a valid proof of knowledge of the corresponding secret key is provided.

Failure of any check causes the transaction to be rejected.

Effect. The `ConfidentialMPTConvert` transaction irreversibly moves value from public to confidential form while preserving full public accountability of total supply. It is the only transaction type that increases COA. Once value enters confidential circulation, its aggregate amount remains publicly tracked via COA, even though its distribution across individual holders—including the issuer’s mirror account—remains hidden.

Transaction Fields. A `ConfidentialMPTConvert` transaction includes the fields summarized in Table 3.

6.2 ConfidentialMPTSend: Confidential Transfer

The `ConfidentialMPTSend` transaction performs a confidential transfer of MPT value between two non-issuer accounts while keeping the transferred amount hidden. The sender’s confidential spending balance is debited, and the receiver’s confidential inbox balance is credited. The receiver may later consolidate incoming funds into their spending balance using `ConfidentialMPTMergeInbox`.

State Transition. Let A denote the sender account and B the receiver account. Upon successful validation, the ledger applies the following deterministic updates.

Sender account A:

$$\begin{aligned} \text{CB}_S(A) &\leftarrow \text{CB}_S(A) \ominus \text{Enc}_A(m), \\ \text{CB_S_Version}(A) &\leftarrow \text{CB_S_Version}(A) + 1, \\ \text{Enc}_I(A) &\leftarrow \text{Enc}_I(A) \ominus \text{Enc}_I(m). \end{aligned}$$

Receiver account B:

$$\begin{aligned} \text{CB}_{\text{IN}}(B) &\leftarrow \text{CB}_{\text{IN}}(B) \oplus \text{Enc}_B(m), \\ \text{Enc}_I(B) &\leftarrow \text{Enc}_I(B) \oplus \text{Enc}_I(m). \end{aligned}$$

If an auditor policy is active, corresponding auditor-encrypted ciphertexts are updated analogously. No public balances are modified, and the global supply fields OA and COA remain unchanged.

Commitments and Ciphertexts. Confidential balance updates on-ledger are performed using EC–ElGamal ciphertexts. Bulletproof range proofs, however, operate over Pedersen commitments whose generators are independent of the ElGamal encryption keys. Accordingly, each **ConfidentialMPTSend** transaction includes the following Pedersen commitments:

$$PC_m = mG + r_mH, \quad PC_b = bG + r_bH,$$

where PC_m commits to the transferred amount m , and PC_b commits to the sender’s confidential spending balance b at the time of the transfer. Within the ZKP, the prover derives a Pedersen commitment to the post-transfer remainder balance as

$$PC_{\text{rem}} := PC_b - PC_m = (b - m)G + (r_b - r_m)H,$$

and proves in zero knowledge that this derived commitment encodes a non-negative value. The commitment PC_{rem} is not transmitted or stored on-ledger; it is computed by the verifier from PC_b and PC_m during proof verification. The commitment PC_b is required to enable a ZKP of balance sufficiency without revealing the sender’s confidential balance.

Zero-Knowledge Proofs. Each **ConfidentialMPTSend** transaction carries a ZKP bundle π_{send} establishing correctness of the confidential transfer without revealing the transferred amount m or the sender’s confidential balance. Concretely, the proof establishes:

1. *Ciphertext equality*: all ElGamal ciphertexts included in the transaction encrypt the same hidden plaintext amount m using shared encryption randomness to ensure cross-ciphertext consistency.
2. *Amount Equality Proof (ElGamal–Pedersen)*: the Pedersen commitment PC_m commits to the same amount m as the sender’s ElGamal ciphertext $\text{Enc}_A(m)$.
3. *Balance Equality Proof (ElGamal–Pedersen)*: the Pedersen commitment PC_b commits to the same confidential balance value b as the sender’s encrypted confidential spending balance prior to the transfer.
4. *Range and sufficiency*: using an aggregated Bulletproof, the prover proves both that

$$0 \leq m < 2^{64} \quad \text{and} \quad 0 \leq b - m < 2^{64},$$

where the second statement is proven over the derived commitment $PC_{\text{rem}} := PC_b - PC_m$.

5. *Replay protection*: the proof transcript is bound to the sender’s current $\text{CB_S_Version}(A)$ via the `TransactionContextID`.

All proofs are non-interactive and are verified by validators prior to applying any ledger state updates.

Validation Rules. Before applying the state transition, validators perform the following checks:

- confidential transfers are enabled for the issuance;
- the sender is not the issuer account;
- sender and receiver addresses are distinct and valid ledger accounts;
- the receiver’s ElGamal public key is already registered;
- all submitted ciphertexts and Pedersen commitments are well-formed secp256k1 group elements;
- any auditor ciphertext conforms to the active auditor policy; and
- the ZKP bundle verifies successfully.

Failure of any check causes the transaction to be rejected.

Effect and Leakage. The **ConfidentialMPTSend** transaction redistributes value entirely within confidential circulation. Observers learn only the identities of the sender and receiver and the fact that a confidential transfer occurred. The transferred amount, individual balances, and the distribution of confidential supply remain hidden.

Transaction Fields. A **ConfidentialMPTSend** transaction includes the fields summarized in Table 4.

6.3 ConfidentialMPTMergeInbox: Inbox Consolidation

The **ConfidentialMPTMergeInbox** transaction consolidates a holder's confidential inbox balance into their confidential spending balance. It homomorphically moves the entire inbox balance into the spending balance and then resets the inbox to a canonical encryption of zero. This transaction ensures that ZKPs for confidential transfers always reference a stable spending balance and prevents proof staleness caused by incoming transfers. The merge operation applies uniformly to all holders. It is typically executed prior to initiating new confidential transfers, but may be invoked at any time. No value is revealed by this transaction, and it does not modify OA.

State Transition. For an account A , the ledger applies the following deterministic state updates:

$$\begin{aligned} \text{CB}_S(A) &\leftarrow \text{CB}_S(A) \oplus \text{CB}_{\text{IN}}(A), \\ \text{CB}_{\text{IN}}(A) &\leftarrow \text{EncZero}(A), \\ \text{CB_S_Version}(A) &\leftarrow \text{CB_S_Version}(A) + 1. \end{aligned}$$

All updates are performed homomorphically. The canonical encrypted zero is defined deterministically as an ElGamal encryption of 0 under the account's public key. Concretely, a scalar

$$r \leftarrow H(\text{"EncZero"} \parallel A \parallel I \parallel C) \bmod q$$

is derived from a domain-separated hash, where n is the curve order, A is the account address, and (I, C) identify the issuance. The ciphertext is then defined as

$$\text{EncZero}(A) = (R = rG, S = r \cdot \text{pk}_A).$$

This construction is deterministic across validators and requires no randomness.

Zero-Knowledge Proofs. No ZKP is required for **ConfidentialMPTMergeInbox**. The transaction performs no user-supplied value updates and moves exactly the inbox balance already recorded in ledger state. Because the operation is fully determined by on-ledger data, there is no opportunity for misreporting or value manipulation.

Validation Rules. Before applying the merge, validators perform the following checks:

- the referenced **MPTokenIssuance** and **MPToken** objects exists and supports confidential transfers;
- no user-supplied ciphertexts are present; and
- the transaction type and context are correctly domain-separated.

Failure of any check causes the transaction to be rejected.

Rationale and Safety. The merge operation enforces several critical safety properties. First, it eliminates proof staleness by ensuring that outgoing transfers always reference a stable spending balance. Second, incrementing the spending balance version invalidates any in-flight proofs bound to a prior state, preventing replay attacks. Third, resetting the inbox to a canonical encrypted zero guarantees that the inbox field always contains a well-formed ciphertext, preserving cryptographic correctness for subsequent operations.

6.4 ConfidentialMPTConvertBack: Confidential to Public

The **ConfidentialMPTConvertBack** transaction converts confidential MPT value back into a publicly visible (plaintext) MPT balance. It reveals a plaintext amount m on-ledger and proves that this amount is consistent with a decrement applied to the account's confidential spending balance. At a high level, **ConfidentialMPTConvertBack** decreases the confidential pool tracked by COA since value leaves confidential circulation and re-enters the public accounting domain. Because the converting account remains the owner of the value, OA is unchanged.

State Transition. Let A be the converting account. Upon successful validation, the ledger applies the following deterministic updates:

$$\begin{aligned} \text{CB}_S(A) &\leftarrow \text{CB}_S(A) \ominus \text{Enc}_A(m), \\ \text{Enc}_I(A) &\leftarrow \text{Enc}_I(A) \ominus \text{Enc}_I(m), \\ \text{CB_S_Version}(A) &\leftarrow \text{CB_S_Version}(A) + 1, \\ \text{COA} &\leftarrow \text{COA} - m. \end{aligned}$$

No other confidential or public balance fields are modified, and the outstanding amount OA remains unchanged. Each **ConfidentialMPTConvertBack** transaction includes cryptographic proofs:

1. *Ciphertext correctness for known plaintext.* For each included ElGamal ciphertext (account, issuer, and optional auditor), validators verify that the ciphertext correctly encrypts the publicly revealed amount m under the corresponding public key. Since m is public, this check is performed via direct verification using disclosed encryption randomness, rather than via a ZKP.
2. *Balance equality (ElGamal–Pedersen).* A linkage proof shows that the Pedersen commitment PC_b corresponds to the converting account’s encrypted confidential spending balance prior to the conversion.
3. *Balance sufficiency.* The prover demonstrates that the converting account’s confidential spending balance is at least m by proving, via a Bulletproof range proof, that the derived remainder commitment $PC_{\text{rem}} := PC_b - mG$ encodes a value in the range $[0, 2^{64})$.
4. *Replay protection.* All proofs are transcript-bound to the current $\text{CB_S_Version}(A)$ and to the transaction context, preventing reuse against stale balances.

All proofs are non-interactive and are verified by validators prior to applying any ledger state updates.

Validation Rules. Validators enforce the following checks before applying the state transition:

- the referenced issuance exists and has confidential transfers enabled;
- the revealed amount satisfies $m \geq 0$ and conforms to XRPL amount formatting and token precision rules;
- all required ElGamal ciphertext fields are present and are well-formed secp256k1 group elements;
- for each ciphertext, direct verification using the disclosed encryption randomness confirms correct encryption of the revealed amount m ;
- the ElGamal–Pedersen balance equality proof verifies successfully, linking the committed balance to the encrypted confidential balance;
- the Bulletproof range proof verifies, establishing that the post-conversion remainder balance is non-negative;
- if an auditor ciphertext is provided, an AuditorPolicy must be active and the auditor key must match the policy; and
- all cryptographic proofs are transcript-bound to $\text{CB_S_Version}(A)$ and to the transaction context.

Effect and Accounting. **ConfidentialMPTConvertBack** is one of the two transaction families that decrease COA (the other being **ConfidentialMPTClawback**). For holders, the transaction moves value from confidential to public form without changing OA. In all cases, observers learn only the revealed conversion amount m and the affected accounts; the distribution of the remaining confidential supply across other holders remains hidden.

Transaction Fields. A **ConfidentialMPTConvertBack** transaction includes the fields summarized in Table 5.

Leakage Discussion. The **ConfidentialMPTConvertBack** transaction necessarily reveals the plaintext conversion amount m , since value is moved from confidential to public form. This disclosure is intentional and mirrors the visibility of issuance and redemption events in standard XLS-33 semantics. However, the protocol is designed so that this revelation does not compromise the privacy of individual confidential balances or the distribution of remaining confidential supply. From a global perspective, observers can infer only the net change in confidential circulation. Specifically, a conversion back of amount m reduces the public field COA by m , allowing any observer to track how much value has exited confidential circulation in aggregate. These effects are limited to global accounting and do not reveal how the remaining confidential supply is distributed across other holders.

At the account level, no additional information is leaked. ElGamal encryptions are randomized and semantically secure: encryptions of the value 0 are computationally indistinguishable from encryptions of nonzero values. Consequently, after a **ConfidentialMPTConvertBack** transaction, an observer cannot determine whether the converting

account’s confidential balance was fully depleted or whether it retains a nonzero confidential balance. Likewise, observers cannot infer whether any other specific account holds part of the remaining confidential supply.

More generally, while conversions between public and confidential forms reveal the amounts crossing the boundary, all redistribution within confidential circulation remains hidden. The protocol therefore provides a clear and auditable view of total supply movements, while preserving confidentiality of individual balances and transfers. This balance between transparency and privacy is intentional and aligns with the design goal of supporting both institutional auditability and confidential asset management on XRPL.

6.5 ConfidentialMPTClawback: Issuer-Enforced Reclamation

The **ConfidentialMPTClawback** transaction allows the issuer to forcibly reclaim confidential MPT value from a holder. This operation is issuer-only and is fundamentally distinct from ordinary confidential transfers. Because the issuer does not possess the holder’s private ElGamal key, it cannot construct a standard **ConfidentialMPTSend** transaction on the holder’s behalf. Instead, the protocol defines a single privileged transaction that enables verifiable reclamation in one atomic ledger operation. Clawback converts a holder’s entire confidential balance directly into the issuer’s public reserve. Partial clawbacks are not supported. This restriction simplifies verification, prevents ambiguous intermediate states, and ensures that public supply accounting remains consistent and auditable.

Operational Overview. The clawback process proceeds as follows. First, the issuer decrypts the issuer-encrypted mirror ciphertext stored on the holder’s **MPToken** object, revealing the holder’s total confidential balance m . The issuer then submits a **ConfidentialMPTClawback** transaction that reveals m in plaintext and includes a ZKP attesting that m is indeed the value encrypted in the on-ledger issuer ciphertext. Validators verify the proof and, if successful, apply the clawback state transition atomically.

State Transition. Let A denote the holder being clawed back. Upon successful validation, the ledger applies the following updates:

$$\begin{aligned} \text{CB}_S(A) &\leftarrow \text{EncZero}(A), \\ \text{CB}_{IN}(A) &\leftarrow \text{EncZero}(A), \\ \text{Enc}_I(A) &\leftarrow \text{EncZero}_I(A), \\ \text{CB_S_Version}(A) &\leftarrow \text{CB_S_Version}(A) + 1, \\ \text{OA} &\leftarrow \text{OA} - m, \\ \text{COA} &\leftarrow \text{COA} - m. \end{aligned}$$

All confidential balances associated with the holder are reset to canonical encryptions of zero. The issuer’s public reserve is increased by the revealed amount m , and both global accounting fields are updated accordingly. This transaction is atomic: either all updates are applied, or none are.

ZKP (Issuer Secret-Key Instantiation). Each **ConfidentialMPTClawback** transaction includes an Ciphertext–Amount Consistency Proof instantiated using the issuer’s ElGamal secret key. This proof establishes that the plaintext amount m , which is revealed on-ledger as part of the clawback, is exactly the value encrypted in the issuer-encrypted mirror ciphertext stored for the holder. Unlike **ConfidentialMPTConvert** and **ConfidentialMPTConvertBack**, the issuer does not possess the original encryption randomness used to form the ciphertext. Therefore, ciphertext correctness cannot be verified via randomness disclosure. Instead, the issuer proves correctness using knowledge of its ElGamal secret key, without revealing that key. Concretely, let $\text{Enc}_I(A) = (C_1, C_2)$ denote the issuer-encrypted confidential balance stored on ledger for holder A , and let $PK_{\text{iss}} = sk_{\text{iss}} \cdot G$ be the issuer’s ElGamal public key with corresponding secret key sk_{iss} . The proof establishes knowledge of sk_{iss} such that:

$$PK_{\text{iss}} = sk_{\text{iss}} \cdot G, \quad C_2 - m \cdot G = sk_{\text{iss}} \cdot C_1.$$

This is implemented as a Chaum–Pedersen proof of equality of discrete logarithms, and proves that the ciphertext $\text{Enc}_I(A)$ encrypts the revealed plaintext value m under the issuer’s public key. The proof reveals no information beyond the correctness of this statement. The proof transcript includes the **TransactionContextID** and is bound to the specific holder and issuance, preventing replay or substitution attacks. No range proof or balance-sufficiency proof is required, since the clawback operation reclaims the holder’s entire confidential balance in a single atomic step.

Validation Rules. Validators enforce the following checks:

- the transaction is signed by the issuer account defined in the issuance;
- the referenced issuance exists and supports confidential transfers and clawback;
- the holder account exists and has a corresponding `MPToken` object;
- the revealed amount m is non-negative and well-formed;
- the issuer-encrypted ciphertext referenced by the transaction matches the on-ledger issuer mirror ciphertext and is a valid `secp256k1` group element;
- the optimized ciphertext-amount consistency proof (issuer secret-key instantiation) verifies successfully; and
- the proof transcript is correctly bound to the transaction context, preventing replay or substitution.

Failure of any check causes the transaction to be rejected.

Effect and Accounting. `ConfidentialMPTClawback` is the only transaction that simultaneously: (i) reveals a holder’s confidential balance, (ii) reduces both `OA` and `COA`, and (iii) transfers value directly into the issuer’s public reserve. This operation preserves public supply soundness while enabling issuer-enforced recovery for compliance, regulatory, or emergency scenarios.

Leakage Discussion. Clawback intentionally reveals the total confidential balance of the affected holder. This disclosure is unavoidable by design and reflects the issuer’s explicit exercise of reclaim authority. Beyond the revealed amount and the identity of the holder, no additional information is leaked. In particular, the confidential balances of all other holders remain hidden. Observers can verify that global supply invariants are maintained, but cannot infer any information about the distribution of remaining confidential supply across accounts.

Transaction Fields. A `ConfidentialMPTClawback` transaction includes the fields summarized in Table 6.

6.6 Freeze Semantics for Confidential MPTs

XLS-33 allows an issuer to freeze or unfreeze a holder’s public MPT balance using the `MPTokenIssuanceSet` transaction. Rather than introducing a new confidential freeze transaction, Confidential MPTs extend the semantics of `MPTokenIssuanceSet` with a deterministic confidential side effect. This preserves existing issuer control mechanisms while ensuring that confidential balances cannot be used to bypass a freeze.

Conceptually, a freeze applies to the *holder’s ability to spend*, regardless of whether their balance is public or confidential. Accordingly, when a holder’s public MPT balance is frozen, their confidential spending capability must also be frozen.

Confidential Freeze State. Each holder’s `MPToken` object includes a boolean flag

$$\text{ConfidentialBalanceFrozen} \in \{\text{true}, \text{false}\},$$

which indicates whether confidential spending from that account is permitted. This flag has no effect on balance representation or encryption; it acts purely as an authorization control enforced by validators.

Validator Logic Extension. When processing a valid `MPTokenIssuanceSet` transaction that modifies the freeze status of a holder’s public MPT balance, validators apply the following additional logic *after* all standard XLS-33 checks have completed successfully:

- If the transaction locks (freezes) the holder’s public MPT balance, then

$$\text{ConfidentialBalanceFrozen} \leftarrow \text{true}.$$

- If the transaction unlocks (unfreezes) the holder’s public MPT balance, then

$$\text{ConfidentialBalanceFrozen} \leftarrow \text{false}.$$

Transaction	Verified Statement	Mechanism	Type
Convert	Ciphertexts encrypt public m	ElGamal check using disclosed r	Deterministic
	Confidential key ownership (first use)	Schnorr secret-key PoK	ZKP
Send	All ciphertexts hide same m	Shared-randomness plaintext equality	ZKP
	PC_m matches encrypted amount	ElGamal–Pedersen equality	ZKP
	PC_b matches encrypted balance	ElGamal–Pedersen equality	ZKP
	$m \geq 0, b - m \geq 0$	Aggregated Bulletproof range proof	ZKP
ConvertBack	Ciphertexts encrypt public m	ElGamal check using disclosed r	Deterministic
	PC_b matches encrypted balance	ElGamal–Pedersen equality	ZKP
	$b - m \geq 0$	Bulletproof range proof	ZKP
Clawback	Issuer ciphertext decrypts to m	Issuer secret-key equality proof	ZKP

Table 1: Cryptographic verification mechanisms required by Confidential MPT transactions.

Effect on Confidential Transactions. If `ConfidentialBalanceFrozen = true` for a holder account A , validators must reject any transaction that attempts to reduce A ’s confidential spending balance for `ConfidentialMPTSend` where A is the sender and `ConfidentialMPTConvertBack` initiated by A . Incoming confidential transfers to the inbox balance (CB_{IN}) remain permitted, as they do not grant spending capability. The holder may also execute `ConfidentialMPTMergeInbox`, since it does not reduce the total confidential balance and has no effect on supply or transferability.

Accounting and Privacy Properties. Freezing or unfreezing a holder has no effect on OA and COA. No amounts are revealed, no ciphertexts are changed, and no ZKPs are required. Observers learn only that the issuer has exercised (or released) a freeze on a specific holder, which is already visible under standard XLS-33 semantics.

Rationale and Safety. Reusing `MPTokenIssuanceSet` ensures that confidential and public balances are governed by a single freeze authority. This prevents holders from evading issuer-imposed restrictions by shifting value into confidential form. Because the freeze logic affects only authorization and not balance representation, it introduces no additional cryptographic assumptions and does not weaken confidentiality guarantees for unfrozen accounts.

6.7 Summary of Cryptographic Proof Requirements

This section summarizes the cryptographic verification mechanisms required by each Confidential MPT transaction type. Table 1 provides a view of (i) which correctness statements must be verified for each transaction, (ii) which proof mechanisms are used, and (iii) which checks are deterministic versus zero-knowledge.

7 Auditability and Compliance

A central challenge in deploying confidential assets on a public ledger is ensuring that confidentiality does not undermine regulatory oversight, supply integrity, or independent verification. Confidential MPTs address this challenge by separating public supply accountability from selective balance visibility, and by enforcing this separation cryptographically at the protocol level. This section describes the audit and compliance framework of Confidential MPTs. The protocol adopts a trust-minimized, on-chain selective disclosure model as its primary audit mechanism,

while remaining architecturally capable of supporting issuer-mediated audit models in future deployments.

Audit Objectives and Public Supply Integrity. The audit framework is designed to satisfy several core objectives. First, supply correctness must remain fully public and trustless. Any observer must be able to verify that the invariant $OA \leq MA$ holds at all times, without access to private keys or confidential balances. Second, the protocol must support selective visibility of balances for authorized auditors, without revealing those balances to validators or other parties. Third, auditability must not rely on issuer honesty: authorized auditors should be able to decrypt balances independently, using cryptographic keys defined on-ledger, unless an explicitly trust-based audit model is chosen. Finally, disclosure should be minimal and purpose-driven, and the design must remain forward-compatible with evolving compliance requirements.

These objectives are realized first and foremost through public, trustless supply accounting. Even with confidential balances, supply correctness remains fully visible. The total circulating supply is defined as

$$OA = \text{PublicOutstanding} + COA,$$

where **PublicOutstanding** is computed directly from visible ledger balances and **COA** is a plaintext ledger field updated only by conversion-style transactions that explicitly reveal their amounts. Because confidential transfers never affect **OA** or **COA**, any observer can independently verify $OA \leq MA$ without decrypting any confidential balances. This preserves XLS-33 supply transparency while enabling confidential circulation.

Cryptographic Audit Architecture. Confidential MPTs represent each confidential balance using multiple ElGamal ciphertexts that encrypt the same underlying plaintext amount m :

$$(\text{Enc}_{\text{pk}_H}(m), \text{Enc}_{\text{pk}_I}(m), \text{Enc}_{\text{pk}_A}(m)),$$

where pk_H is the holder’s key, pk_I is the issuer’s key, and pk_A is an auditor key when an **AuditorPolicy** is active. ZK equality proofs enforce that all ciphertexts remain synchronized and encrypt the same value, preventing divergence between different encrypted views of the same balance.

On-Chain Selective Disclosure (Cryptographic Auditor Model). The primary audit mechanism used by the protocol is a trust-minimized, fully cryptographic auditor model based on on-chain selective disclosure. When an **AuditorPolicy** is active, each confidential balance includes a ciphertext encrypted under the auditor’s public key. An auditor holding the corresponding private key can independently decrypt balances, verify balance transitions, and reconcile confidential circulation against the public **COA** field. No interaction with the issuer or holder is required. Cryptographic guarantees are enforced as follows. Equality proofs ensure that auditor ciphertexts match the holder and issuer views. Auditors cannot forge or modify balances, as all state transitions are validated by ZKPs and deterministic ledger rules. For all other observers, auditor ciphertexts are indistinguishable from random ElGamal ciphertexts and leak no information.

8 Security and Privacy Analysis

This section analyzes the security properties of Confidential MPTs under the adversarial model of a public, permissionless ledger. We examine confidentiality guarantees, integrity of confidential balances, soundness of the ZKP system, correctness of supply tracking, and protocol-level defenses against replay, manipulation, and denial-of-service attacks.

We assume an adversary with full visibility of all ledger data, including ciphertexts and ZKPs, and the ability to generate arbitrary transactions and submit them to the network. The adversary may corrupt or collude with an arbitrary subset of clients, but does not control a majority of validators. All parties are computationally bounded by standard cryptographic hardness assumptions for secp256k1. Validators are assumed honest-but-curious: they faithfully execute protocol rules but may attempt to infer confidential information from transaction data. Holders, issuers, and auditors may behave arbitrarily except where restricted by protocol invariants and cryptographic verification. We assume all ZKPs used in the protocol are sound and zero-knowledge in the random oracle model and the underlying ledger consensus protocol ensures that accepted transactions reflect deterministic validator execution.

8.1 Confidentiality Guarantees

All confidential balances and transfer amounts are represented as EC–ElGamal ciphertexts over secp256k1. Security relies on the Decisional Diffie–Hellman (DDH) assumption.

Definition 8.1 (IND-CPA Confidentiality Game). *Let $\mathcal{A}$ be a probabilistic polynomial-time adversary. The game between challenger $\mathcal{C}$ and $\mathcal{A}$ proceeds as follows:*

1. $\mathcal{C}$ generates (sk, pk) .
2. $\mathcal{A}$ outputs two equal-length messages m_0, m_1 .
3. $\mathcal{C}$ samples a random bit $b \leftarrow \{0, 1\}$ and returns $C = \text{Enc}_{pk}(m_b)$.
4. $\mathcal{A}$ outputs a guess b' .

The advantage of $\mathcal{A}$ is defined as

$$\text{Adv}_{\mathcal{A}}^{\text{IND}} = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

Theorem 8.1 (Confidentiality under DDH). *Under the DDH assumption in secp256k1, EC–ElGamal encryption used in Confidential MPTs is IND-CPA secure. Formally, for any PPT adversary $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\text{IND}}$ is negligible.*

Proof Sketch. ElGamal encryption over elliptic curves is IND-CPA secure under DDH. All confidential balances and transfer amounts are encrypted using fresh randomness per transaction. Shared randomness is restricted to encryptions of the same plaintext within a single transaction and is enforced via equality proofs, revealing no information beyond plaintext equality. $\square$

It follows that holders’ confidential balances (CB_S and CB_N) are computationally indistinguishable from random group elements. Observers cannot determine whether a balance is zero or positive, nor infer transfer amounts or distribution across accounts. The only intentional confidentiality boundary crossings occur during CONVERT, CONVERTBACK, and CLAWBACK, where amounts are explicitly revealed to preserve XLS-33 supply semantics.

8.2 Integrity of Confidential Balances and Proof Soundness

A core security objective is preventing divergence between the multiple ciphertexts representing a single confidential balance. For any amount m , the holder-, issuer-, and auditor-encrypted ciphertexts

$$\text{Enc}_{pk_H}(m), \text{Enc}_{pk_I}(m), \text{Enc}_{pk_A}(m)$$

are required to be linked via zero-knowledge equality proofs. These proofs guarantee that all encrypted representations encode the identical plaintext value, preventing fabrication of inconsistent encrypted views.

Any transaction that decreases a confidential balance must provide a range proof establishing that the debited amount a satisfies

$$0 < a \leq \text{CB}_S,$$

without revealing either a or the current balance. Because balance updates are performed via homomorphic ciphertext operations, this proof ensures that no overspending, negative balance, or algebraic inconsistency can occur without violating proof soundness.

All ZKPs include a shared transaction context as an input to Fiat–Shamir challenge generation, derived from

$$(\text{TxType}, \text{Account}, \text{MPTokenIssuanceID}, \text{SequenceOrTicket}, \text{TxSpecific}),$$

thereby preventing replay and cross-transaction substitution. When applicable, TxSpecific includes CB_S_Version , which binds the proof to a specific confidential spending-balance state. Any state-changing operation that modifies CB_S , such as MERGEINBOX, increments this version and invalidates previously generated proofs that depend on the prior spending state. Under the soundness of the underlying ZKPs and the hardness of the discrete logarithm problem, an adversary cannot forge transfers, inflate balances, or misrepresent encrypted values without detection.

8.3 Balance Soundness (No Inflation)

We formalize balance integrity via a game that captures the inability of any adversary to increase confidential balances beyond what is implied by valid ledger transitions.

Definition 8.2 (Balance Soundness Game). *Let $\mathcal{A}$ be a PPT adversary interacting with a ledger challenger $\mathcal{C}$ that maintains ledger state and enforces all Confidential MPT validation rules (including signature checks, curve checks, transcript binding, and ZKP verification). The game proceeds as follows:*

1. $\mathcal{C}$ initializes the ledger with protocol parameters and an initial set of accounts and token issuances. The initial ledger state is public to $\mathcal{A}$.
2. $\mathcal{A}$ adaptively submits a sequence of transactions. For each submitted transaction, $\mathcal{C}$ applies the protocol's deterministic validation rules. If valid, the transaction is applied and the ledger state is updated; otherwise it is rejected and the state remains unchanged.
3. $\mathcal{A}$ wins if it reaches a ledger state in which there exists an account whose confidential spending balance CB_S (for some issuance) has increased by an amount that is not attributable to any valid supply-modifying operation (i.e., `CONVERT`, `CONVERTBACK`, or `CLAWBACK`) and is inconsistent with the homomorphic effect of prior accepted transactions and their associated proofs.

Let $\Pr[\text{Win}_{\text{Bal}}(\mathcal{A})]$ denote $\mathcal{A}$'s winning probability in the above game.

Theorem 8.2 (Balance Soundness). *Assume (i) soundness of the ZKPs used by Confidential MPTs in the random oracle model, and (ii) hardness of the discrete logarithm problem in the secp256k1 group. Then for any PPT adversary $\mathcal{A}$, $\Pr[\text{Win}_{\text{Bal}}(\mathcal{A})]$ is negligible.*

Proof Sketch. Any successful inflation requires producing a transaction that is accepted by validators while effecting a net increase in a confidential balance inconsistent with the permitted state transition rules. This implies at least one of: (i) violating the range proof that enforces $0 < a \leq CB_S$, (ii) breaking ciphertext consistency enforced by equality proofs, or (iii) breaking ciphertext-commitment linkage required for range-proof soundness. Each case yields a violation of ZKP soundness or a discrete-log-type assumption. $\square$

8.4 Supply Correctness

Supply correctness is a protocol-level invariant enforced using only plaintext ledger fields. We formalize it as an inability to reach a state violating the XLS-33 supply bound.

Definition 8.3 (Supply Violation Game). *Let $\mathcal{A}$ be any (computationally unbounded) adversary interacting with a ledger challenger $\mathcal{C}$ that executes the Confidential MPT validation rules exactly. The game proceeds as follows:*

1. $\mathcal{C}$ initializes the ledger with a token issuance having maximum supply `MaxAmount`.
2. $\mathcal{A}$ adaptively submits transactions; $\mathcal{C}$ accepts and applies a transaction iff it satisfies all deterministic validation checks.
3. $\mathcal{A}$ wins if it reaches a ledger state where

$$OA > \text{MaxAmount},$$

where OA is the issuance's plaintext outstanding amount as maintained by the ledger.

Let $\Pr[\text{Win}_{\text{Sup}}(\mathcal{A})]$ denote $\mathcal{A}$'s winning probability.

Theorem 8.3 (Supply Correctness). *Assuming validators execute the protocol's deterministic rules correctly, for any adversary $\mathcal{A}$, $\Pr[\text{Win}_{\text{Sup}}(\mathcal{A})] = 0$.*

Proof Sketch. The ledger accepts a transaction only if it preserves the invariant $OA \leq \text{MaxAmount}$ under plaintext arithmetic. Confidential transfers do not modify OA . The only transactions that can change OA (and/or COA) are `CONVERT`, `CONVERTBACK`, and `CLAWBACK`, all of which update the relevant plaintext supply fields deterministically and are subject to the same invariant check. Therefore a violating state is unreachable. $\square$

8.5 Replay Resistance

Replay resistance ensures that a proof valid for one ledger snapshot cannot be reused in a different ledger state.

Definition 8.4 (Replay Game). *Let $\mathcal{A}$ be a PPT adversary and $\mathcal{C}$ a ledger challenger that enforces all validation rules. The game proceeds as follows:*

1. $\mathcal{C}$ provides $\mathcal{A}$ with a valid transaction tx and associated proof(s) π that verify under ledger context ctx , where ctx includes the fields hashed into `TransactionContextID` (e.g., account identifiers, issuance ID, sequence/ticket, transaction domain, and when applicable `CB_S_Version`).
2. $\mathcal{A}$ outputs a modified transaction tx' that reuses π in a different ledger context $\text{ctx}' \neq \text{ctx}$ (e.g., after a state change that increments `CB_S_Version`).
3. $\mathcal{A}$ wins if tx' is accepted by the ledger.

Let $\Pr[\text{Win}_{\text{Rep}}(\mathcal{A})]$ denote $\mathcal{A}$'s winning probability.

Theorem 8.4 (Replay Resistance). *Assume the hash function is modeled as a random oracle. Then for any PPT adversary $\mathcal{A}$, $\Pr[\text{Win}_{\text{Rep}}(\mathcal{A})]$ is negligible.*

Proof Sketch. All NIZKs are transcript-bound to `TransactionContextID`, which is a hash of transaction fields and replay-prevention state (sequence/ticket, and when applicable `CB_S_Version`). If $\text{ctx}' \neq \text{ctx}$, then the derived context identifier changes, inducing different Fiat–Shamir challenges. Reusing π therefore causes verification failure except with negligible probability, which would otherwise imply a collision or random-oracle inconsistency. $\square$

8.6 No Double-Spend

We formalize double-spending as the ability to reuse the same confidential spending balance snapshot to authorize multiple outgoing transfers whose combined debits exceed the available balance. Confidential MPTs prevent this by binding sender-controlled spend proofs to the current `CB_S_Version` whenever confidential spending balance is consumed, while XRPL's native `Sequence` or `Ticket` mechanism provides transaction-level replay protection across all transaction types.

Definition 8.5 (Double-Spend Game). *Let $\mathcal{A}$ be a PPT adversary interacting with a ledger challenger $\mathcal{C}$ that maintains ledger state and enforces all Confidential MPT validation rules (including signature checks, curve checks, transcript binding, and ZKP verification). The game proceeds as follows:*

1. $\mathcal{C}$ initializes the ledger and provides $\mathcal{A}$ with public parameters and the initial public ledger state. In particular, $\mathcal{A}$ observes an account A with confidential spending balance CB_S for some issuance and current version $v := \text{CB_S_Version}(A)$.
2. $\mathcal{A}$ adaptively submits transactions. The challenger accepts and applies a transaction iff it satisfies all deterministic validation checks, updating ledger state accordingly.
3. $\mathcal{A}$ wins if it causes the ledger to accept two transactions tx_1 and tx_2 that both debit the same account A , such that:
 - both transactions' spend proofs are transcript-bound to the same version value v (i.e., both prove against the same `CB_S_Version = v`), and
 - the combined debited amount exceeds what can be authorized from the single balance snapshot at version v under the protocol's spending rules.

Let $\Pr[\text{Win}_{\text{DS}}(\mathcal{A})]$ denote $\mathcal{A}$'s winning probability.

Theorem 8.5 (No Double-Spend). *Assume (i) soundness of the ZKPs used by Confidential MPTs in the random oracle model and (ii) correct validator execution of the state transition rule that increments `CB_S_Version` upon any accepted spend from CB_S . Then for any PPT adversary $\mathcal{A}$, $\Pr[\text{Win}_{\text{DS}}(\mathcal{A})]$ is negligible.*

Proof Sketch. Any accepted spend from A must verify a proof bound to the current `CB_S_Version` and, upon acceptance, deterministically increments that version in the ledger state. Hence, after tx_1 is accepted, the ledger’s version changes from v to $v + 1$, and any proof bound to version v becomes stale. For tx_2 to also be accepted while still bound to v , A must either (i) bypass deterministic version updates (violating the ledger execution model), or (ii) produce a proof that verifies under an incorrect context identifier, contradicting transcript binding and ZKP soundness except with negligible probability in the random oracle model. $\square$

8.7 Implementation-Level and Operational Mitigations

In addition to the cryptographic guarantees proven above, Confidential MPTs incorporate several protocol-level defenses. All submitted ciphertexts are validated via curve-membership checks, preventing invalid-curve and malformed-point attacks. ZKPs are transcript-bound to transaction-specific context and sequence information, eliminating replay and cross-transaction reuse. Because confidential amounts are encoded as ElGamal ciphertexts with fresh randomness, front-running and ordering inference do not reveal transfer magnitudes. Denial-of-service attempts based on stale inbox manipulation are mitigated by the split-balance model and the deterministic, proof-free `MERGEINBOX` operation. Although the issuer retains privileged operations such as `CLAWBACK`, these actions are cryptographically constrained. Any clawback requires a proof tying the publicly revealed amount to the issuer-encrypted confidential balance of the holder. Since holder- and issuer-side ciphertexts are linked by equality proofs throughout the account lifecycle, the issuer cannot fabricate or inflate clawback amounts without violating proof soundness.

8.8 Privacy Properties

Beyond balance integrity and replay resistance, Confidential MPTs provide confidentiality guarantees for transferred amounts and confidential balances. This subsection characterizes the precise privacy scope of the protocol. We explicitly describe what information is hidden, what remains publicly visible, and what is intentionally not protected. The goal is to precisely define the privacy scope of the protocol and avoid ambiguous or overstated claims.

Validators are assumed honest-but-curious: they faithfully execute protocol rules and verify ZKPs, but must not learn confidential balances or transfer amounts. Auditors may be designated via on-chain policy and can decrypt auditor-specific ciphertexts, but cannot modify or spend holder funds. Confidential MPTs are designed to provide amount and balance confidentiality while preserving XRPL’s transparent execution and account model. They intentionally do not provide sender anonymity, receiver anonymity, transaction unlinkability, or graph obfuscation. All transaction participants remain publicly identifiable, consistent with standard XLS-33 and XRPL semantics. For all Confidential MPT transactions, the following information is publicly visible on-ledger: transaction type; sender, receiver, issuer, and currency identifiers; ElGamal ciphertexts represented as elliptic curve points; ZKPs and associated metadata; and public supply fields `OA`, `COA`, and `MA`. This visibility ensures deterministic validation, auditability, and compatibility with existing XRPL infrastructure. The following information is cryptographically hidden under the DDH assumption: the amount transferred in `CONFIDENTIALMPTSEND`; per-account confidential spending and inbox balances; the distribution of confidential supply across holders.

Confidential MPTs resist common inference attacks. Because encryptions of zero are indistinguishable from nonzero encryptions, observers cannot determine whether a holder has emptied their confidential balance or whether the issuer’s second account retains funds after redistribution. Equality proofs bind ciphertexts only within a single transaction and do not allow amount correlation across transactions. Observers may infer net changes in confidential circulation only through public `COA` updates during `Convert` and `ConvertBack`, but cannot attribute those changes to specific holders.

Auditor access is explicitly scoped. Auditors can decrypt balances only if designated, can never spend funds, and do not gain visibility beyond what their key permits. The presence of auditor ciphertexts does not weaken privacy for non-auditors, as they remain indistinguishable without the corresponding private key.

The privacy model intentionally preserves XRPL’s transparent execution semantics, including public transaction ordering and visible account relationships. As a result, transaction graphs, sender–receiver relationships, and timing information remain observable. Protection against graph analysis or off-ledger correlation would require additional mechanisms such as stealth addresses or ring signatures, which are outside the scope of this specification.

9 Performance Analysis

This section evaluates the computational, bandwidth, and storage costs of Confidential MPT operations, covering client-side proving, validator-side verification, and ledger storage overhead. We also discuss how these costs scale with adoption and supported amount ranges. All estimates assume EC-ElGamal encryption over `secp256k1`, compressed elliptic-curve points (33 bytes), and Bulletproof range proofs instantiated for 64-bit values. Unless stated otherwise, performance figures correspond to software implementations executed on general-purpose CPU platforms without hardware acceleration.

We report sizes for the auditor-enabled configuration of `ConfidentialMPTSend`, which represents the most general deployment setting. The analysis assumes the shared-randomness ElGamal construction, where a single first ciphertext component is shared across all recipients. Using shared randomness, the transaction contains $C_1 = rG$ and $C_{2,i} = mG + r \cdot Pk_i$ for each recipient public key. For sender, destination, issuer, and auditor, this yields one shared C_1 point (33 bytes) and four distinct $C_{2,i}$ points (132 bytes), for a total ciphertext size of $5 \times 33 = 165$ bytes. To support zero-knowledge verification, the transaction additionally includes Pedersen commitments to: (i) the transfer amount m , and (ii) the sender’s confidential spending balance prior to transfer. Each commitment occupies one compressed elliptic-curve point, giving $2 \times 33 = 66$ bytes. The transaction carries an aggregated Bulletproof establishing $0 \leq m < 2^{64}$ and $0 \leq b - m < 2^{64}$, where b denotes the sender’s confidential balance before transfer. As described in [BBB⁺18], an aggregated proof for two 64-bit values transmits 18 group elements and 5 scalars, yielding $18 \times 33 + 5 \times 32 = 754$ bytes. The next one, consistency across ciphertexts and the amount commitment is established through a same-plaintext equality proof. For N ciphertexts, this proof contains $(1 + N)$ elliptic-curve points and 2 scalars. For the auditor-enabled case ($N = 4$), this yields $5 \times 33 + 2 \times 32 = 229$ bytes. Finally, balance correctness requires two Pedersen-ElGamal linkage proofs relating the sender’s confidential balance ciphertext to its balance commitment, adding approximately $2 \times 195 = 390$ bytes. Combining all components, an auditor-enabled confidential send transaction requires approximately

$$\begin{aligned} &165 \text{ bytes (shared-randomness ElGamal ciphertexts)} \\ &+ 66 \text{ bytes (Pedersen commitments)} \\ &+ 754 \text{ bytes (aggregated Bulletproof)} \\ &+ 229 \text{ bytes (same-plaintext proof)} \\ &+ 390 \text{ bytes (Pedersen-ElGamal linkage proofs)} \\ &= 1604 \text{ bytes} \approx 1.6 \text{ KB.} \end{aligned}$$

Without auditor support, the transaction is smaller by one ciphertext element and the corresponding same-plaintext proof overhead.

Computation and storage costs.

The dominant client-side cost of confidential transactions is Bulletproof generation; ElGamal encryption and auxiliary proof construction contribute only minor additional overhead relative to the range proof itself. We developed a reference implementation of the cryptographic primitives used in this protocol in the `mpt-crypto` library.¹ The current implementation prioritizes correctness and modularity and does not yet include low-level optimizations such as optimized multi-scalar multiplication, generator precomputation, or batched field inversion. Using this reference implementation, generation of an aggregated Bulletproof for two 64-bit values currently requires approximately 37.6 ms while verification requires approximately 19.3 ms on commodity CPU hardware. The measured proof size is 754 bytes, matching the analytical size derived above. These timings are expected to decrease substantially in optimized production implementations, where multi-scalar multiplication dominates prover execution.

`ConfidentialMPTMergeInbox` incurs no proving overhead because it is deterministic and requires no zero-knowledge proof. `CONFIDENTIALMPTConvert` includes an equality proof over the revealed amount, while `CONFIDENTIALMPTConvertBack` has proving cost comparable to `CONFIDENTIALMPTSend` because it additionally requires a range proof and confidential balance sufficiency proof. Validator execution is likewise dominated by Bulletproof verification together with elliptic-curve point validation. Even in the current reference implementation, these costs remain within XRPL validator execution budgets and do not constitute a throughput bottleneck.

¹<https://github.com/XRPLF/mpt-crypto>

Component	Current modular (bytes)	Amount-unified ElGamal (bytes)	Twisted ElGamal (bytes)
Shared-randomness ciphertexts	165	165	165
Amount commitment	33	33	—
Balance commitment	33	33	—
Aggregated Bulletproof	754	754	754
Same-plaintext proof	229	—	229
Unified amount proof (includes PC_m)	—	262	—
Amount linkage proof	195	—	—
Balance linkage / DDH proof	195	195	98
Refreshed remainder ciphertext	—	—	66
Total	1604	1442	1312

Table 2: Proof size comparison for auditor-enabled confidential transfer.

Ledger storage overhead remains modest. Each holder’s **MPToken** object stores two confidential balance ciphertexts (spending and inbox), an issuer-encrypted mirror ciphertext, an optional auditor-encrypted ciphertext, the holder’s ElGamal public key, a version counter, and small metadata fields. In total, this requires approximately 240–306 bytes per holder, depending on whether auditor support is enabled. Because XRPL stores only resulting ledger state rather than transaction history, storage remains stable over time and scales primarily with the number of participating holders.

Transaction-size optimizations and scalability trade-offs.

At the transaction level, the dominant overhead arises from cryptographic proof material rather than ledger growth. The current protocol uses a modular proof composition in which ciphertext consistency and balance correctness are established through separate proof components. In this baseline construction, the same-plaintext proof establishes that all recipient ciphertexts encrypt the same transfer amount under shared randomness, while two independent Pedersen–ElGamal linkage proofs verify consistency between the amount commitment and transfer ciphertexts, and between the sender’s confidential balance ciphertext and its balance commitment.

A first optimization is an amount-unified ElGamal construction, where the amount commitment PC_m is absorbed directly into the same-plaintext proof. Instead of proving ciphertext equality and commitment linkage separately, the proof is extended with one additional group relation showing that the same plaintext m and shared randomness r simultaneously define all recipient ciphertexts and the Pedersen commitment. This removes one independent Pedersen–ElGamal linkage proof, while the sender’s balance commitment still requires a separate linkage proof because its blinding factor remains distinct. A second optimization is a twisted ElGamal variant, where the plaintext is embedded directly into the ciphertext structure so that the transfer amount commitment becomes implicit in the ciphertext itself, as also explored in auditable confidential payment systems such as PGC [CMTA20]. In this case, explicit amount commitment linkage disappears, and balance correctness is verified using a compact proof together with a refreshed remainder ciphertext. Future optimizations can therefore reduce transaction size from the current modular construction toward approximately 1.3–1.4 KB, while preserving verifier transparency. Table 2 summarizes the approximate transaction size of an auditor-enabled confidential transfer under the current modular design and two representative proof-level optimization strategies. From a scalability perspective, confidential MPTs remain practical for account-based ledger deployment: ledger growth is linear in participating holders, while proof size remains fixed per transaction and independent of transaction history.

10 Limitations and Future Extensions

While Confidential MPTs provide strong confidentiality guarantees for balances and transfer amounts, the current design intentionally prioritizes compatibility, auditability, and deployability over maximal privacy. This section summarizes the principal limitations of the protocol and outlines directions for future extensions.

Privacy scope and transparency. The privacy guarantees of Confidential MPTs are deliberately scoped to preserve XRPL’s transparent, account-based execution model. The protocol does not provide identity anonymity

or transaction unlinkability. Sender and receiver account identifiers remain public for all transactions, and repeated interactions between the same accounts are linkable over time. As a result, transaction graphs, timing information, and account relationships remain visible to observers, enabling graph analysis and frequency-based inference.

These limitations are a conscious design choice rather than a cryptographic constraint. Achieving stronger privacy properties—such as receiver anonymity, sender ambiguity, or graph obfuscation—would require additional layers such as stealth addresses, ring signatures, mix-based routing, or shielded pools. While these techniques are compatible with the underlying confidential balance model, they would significantly alter XRPL’s execution semantics and validator determinism. Accordingly, they are excluded from the present specification and are better treated as optional extensions in future protocol versions.

Operational and performance trade-offs. Confidential MPTs introduce additional operational complexity relative to standard MPTs. Issuers must manage a designated second account to seed confidential circulation, holders must periodically merge inbox balances to maintain a spendable state, and wallets are required to manage ElGamal key material and construct ZKPs. From a performance perspective, confidential transactions incur higher bandwidth and computational costs than plaintext transfers due to ciphertexts and Bulletproof-based range proofs. Although these costs remain well within XRPL’s validator execution budget, they reduce throughput relative to non-confidential transactions. Future work may explore proof aggregation, batching strategies, and protocol-level optimizations that amortize proving costs, as well as wallet-side automation and heuristics that reduce merge frequency and simplify the user experience.

Compliance and audit expressiveness. The current protocol supports a single on-chain auditor via an `AuditorPolicy`. This model is sufficient for many institutional and regulatory use cases but limits expressiveness. The design does not natively support multiple concurrent auditors, conditional or selective disclosure policies, or fine-grained access control. Auditor visibility is binary: an auditor either has full read access to balances or none at all.

Future extensions may consider threshold encryption, multi-auditor ciphertext sets, time-limited audit keys, or policy-based access control mechanisms. These enhancements can be layered on top of the existing multi-ciphertext architecture without modifying core ledger invariants, but they introduce additional cryptographic, operational, and governance complexity.

Cryptographic agility and formal assurance. Confidential MPTs rely on EC–ElGamal encryption and discrete-logarithm-based ZKPs over `secp256k1`, which are not quantum-resistant. Long-term cryptographic agility will require exploring post-quantum confidential value commitments, quantum-safe range proof systems, and migration paths for existing confidential balances that preserve ledger consistency and auditability.

One possible migration path is issuer-side key rotation with lazy re-encryption. Under this approach, the issuer updates its encryption public key in the `MPTokenIssuance` object and increments an associated key epoch. Existing issuer-encrypted mirror balances are then migrated lazily: on a holder’s first confidential transaction after rotation, the transaction includes a new issuer ciphertext under the current epoch together with a proof that both old and new ciphertexts encrypt the same hidden balance. Inactive accounts may be migrated through issuer-triggered maintenance transactions that re-encrypt balances while preserving `ConfidentialOutstandingAmount` consistency and clawback capability.

11 Conclusion

This work presented Confidential MPTs, a protocol extension to XRPL that enables confidential balances and transfer amounts while preserving the core semantics, validation rules, and execution model of XLS-33. Confidentiality is achieved using EC–ElGamal encryption and ZKPs, without modifying XRPL’s public accounting model, consensus process, or deterministic validation guarantees. The central contribution of this work is a protocol architecture that reconciles three traditionally competing objectives: (i) confidentiality of individual balances and transfer amounts, (ii) public and trustless enforcement of supply constraints through existing ledger invariants, and (iii) flexible auditability suitable for regulated and institutional deployments. These objectives are realized through the issuer second-account model, the split-balance mechanism for preventing proof staleness, and a multi-ciphertext representation that simultaneously supports holders, issuers, and optional auditors.

Unlike privacy systems that aim to obscure identities or transaction graphs, Confidential MPTs deliberately scope privacy to value confidentiality. Sender and receiver identities remain visible, preserving compatibility with XRPL’s account-based semantics, transaction ordering, and existing compliance workflows. At the same time, validators never learn confidential amounts, and auditors can be granted verifiable read access without weakening public supply integrity. From a systems perspective, this protocol demonstrates that confidential asset transfers can be integrated into a production ledger without redefining supply semantics, introducing trusted execution components, or requiring ledger-wide cryptographic assumptions beyond those already in use. From a cryptographic perspective, it illustrates how equality proofs and compact range proofs can be composed with homomorphic encryption to enforce ledger invariants over encrypted state.

In summary, Confidential MPTs provide a practical, auditable, and deployable approach to confidential value transfer on XRPL. The protocol bridges the gap between privacy-preserving cryptography and real-world tokenization requirements, and offers a blueprint for adding confidentiality to permissionless ledger systems without sacrificing transparency, correctness, or operational viability.

Acknowledgment

We thank David Schwartz, Kenny Lei, Shashwat Mittal, Shawn Xie, Yinyi Qian, and Peter Chen for their valuable feedback, technical discussions, and suggestions that helped improve this work.

References

- [BAMB20] Benedikt Bünz, Shashank Agrawal, Mary Maller, and Dan Boneh. Zether: Towards privacy in a smart contract world. In *Financial Cryptography and Data Security (FC)*, pages 248–267, 2020. IACR ePrint 2019/191.
- [BBB⁺18] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *IEEE Symposium on Security and Privacy*, pages 315–334, 2018.
- [BBS03] Mihir Bellare, Alexandra Boldyreva, and Jessica Staddon. Multi-recipient encryption schemes: Security notions and randomness re-use, 2003. Full version of PKC 2003 paper. Available at <https://cseweb.ucsd.edu/~mihir/papers/bbs.pdf>.
- [BL12] Daniel J. Bernstein and Tanja Lange. Computing small discrete logarithms faster. In *INDOCRYPT*, 2012.
- [BL13] Daniel J. Bernstein and Tanja Lange. Two grumpy giants and a baby. In *ANTS X*, 2013.
- [BS⁺14] Eli Ben-Sasson et al. Zerocash: Decentralized anonymous payments from bitcoin. In *IEEE Symposium on Security and Privacy*, pages 459–474, 2014.
- [BS20] Dan Boneh and Victor Shoup. *A Graduate Course in Applied Cryptography*. 2020. <https://toc.cryptobook.us/>.
- [CCN21] Panagiotis Chatzigiannis, Konstantinos Chalkias, and Valeria Nikolaenko. Homomorphic decryption in blockchains via compressed discrete-log lookup tables. In *DPM/CBT*, 2021.
- [CGS97] Ronald Cramer, Rosario Gennaro, and Berry Schoenmakers. A secure and optimally efficient multi-authority election scheme. In *EUROCRYPT*, volume 1233 of *LNCS*, pages 103–118, 1997.
- [CMTA20] Yi Chen, Xiaofeng Ma, Chao Tang, and Man Ho Au. Pgc: Decentralized confidential payment system with auditability. In *ESORICS*, pages 591–610, 2020.
- [DH76] Whitfield Diffie and Martin Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976.
- [Dia21] Benjamin E. Diamond. Many-out-of-many proofs and applications to anonymous zether. In *IEEE Symposium on Security and Privacy*, 2021.

- [ElG85] Taher ElGamal. A public-key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Transactions on Information Theory*, 31(4):469–472, 1985.
- [GWZ17] Steven D. Galbraith, Peng Wang, and Fangguo Zhang. Computing elliptic curve discrete logarithms with improved baby-step giant-step algorithm. *AIMS Mathematics*, 2017.
- [Kur00] Kaoru Kurosawa. Multi-recipient public-key encryption with shortened ciphertext. In *Public Key Cryptography (PKC 2000)*, volume 1751 of *Lecture Notes in Computer Science*, pages 47–61. Springer, 2000.
- [Lab22] Solana Labs. Confidential balances, 2022. <https://www.solana-program.com/docs/confidential-balances>.
- [Lab23] Aptos Labs. Confidential asset, 2023. <https://aptos.dev/build/smart-contracts/confidential-asset>.
- [MS25] Varun Madathil and Alessandra Scafuro. Prifhete: Achieving full-privacy in account-based cryptocurrencies is possible. In *Advances in Cryptology – ASIACRYPT 2025*, volume 16251 of *Lecture Notes in Computer Science*, pages 199–229. Springer, 2025.
- [NM16] Shen Noether and Adam Mackenzie. Ring confidential transactions. *Ledger*, 1(1), 2016.
- [Pol00] John M. Pollard. Kangaroos, monopoly and discrete logarithms. *Journal of Cryptology*, 2000.
- [Sma16] Nigel P. Smart. *Cryptography Made Simple*. Springer, 2016.
- [Sol24] Solana Program Library. The twisted elgamal encryption scheme, 2024. https://spl.solana.com/assets/files/twisted_elgamal-2115c6b1e6c62a2bb4516891b8ae9ee0.pdf.
- [TLC⁺23] Fan Tang, Gu Ling, Chenyu Cai, et al. Solving small exponential ecdlp in ec-based additively homomorphic encryption. *IEEE Transactions on Information Forensics and Security*, 2023.
- [XRP24] XRPLF. Xls-0033: Multi-purpose tokens, 2024. <https://github.com/XRPLF/XRPL-Standards>.

A Cryptographic Building Blocks

Confidential MPTs rely on efficient cryptographic primitives. The construction is defined over an elliptic curve group G of prime order n defined over a finite field $\mathbb{F}_p$. Scalars are elements of $\mathbb{Z}_n$ and group elements are elliptic curve points. In the current XRPL implementation, these primitives are instantiated using the `secp256k1` elliptic curve, defined over a prime field $\mathbb{F}_p$ with a cyclic subgroup of order $n \approx 2^{256}$. Group elements are encoded in compressed form using 33 bytes, and scalars are encoded as 32-byte integers. We summarize below the cryptographic primitives used in our construction.

Key Generation

Each account participating in confidential MPT transactions generates an ElGamal key pair. The privkey (sk) is the secret key, kept private by the account holder. The pubkey (Pk) is the corresponding public key, which is registered on the XRP Ledger and used by other parties to encrypt amounts destined for that account. The private key is a 32-byte (256-bit) cryptographically secure random number. This sk must be a scalar within the range $[1, n - 1]$, where n is the order of the `secp256k1` curve’s base point G . The public key is derived by performing scalar multiplication of the base point G as $Pk = sk \cdot G$. This Pk is a point on the `secp256k1` curve. The security of the ElGamal scheme relies on the hardness of the Elliptic Curve Discrete Logarithm Problem (ECDLP) and the Elliptic Curve Computational Diffie-Hellman (ECCDH) problem [BS20]. The private key sk must remain confidential. These ElGamal keys are used exclusively for confidential balance encryption and ZKPs, and are distinct from the XRPL account signing keys used to authorize and sign transactions.

ElGamal Encryption

The Confidential MPT protocol uses elliptic-curve ElGamal public key encryption [DH76, ElG85, CGS97] to encrypt confidential amounts under recipient-specific public keys. ElGamal encryption enables additive homomorphism, which is required to support confidential balance updates on-ledger. Let G denote a generator of an elliptic-curve group of prime order q , and let $Pk_i = sk_i \cdot G$ denote the ElGamal public key of recipient i . To encrypt an amount $m \in \mathbb{Z}_q$, the encryptor samples a random scalar $r \xleftarrow{\$} \mathbb{Z}_q$ and computes:

$$\begin{aligned} C_1 &= r \cdot G, \\ C_{2,i} &= m \cdot G + r \cdot Pk_i. \end{aligned}$$

The resulting ciphertext for recipient i is $(C_1, C_{2,i})$.

Shared Randomness Optimization. When encrypting the same amount m for multiple recipients within a single transaction (e.g., sender, receiver, issuer mirror, auditor), the protocol uses a shared encryption randomness r across all such ciphertexts. This results in a common first component $C_1 = rG$ and distinct second components $C_{2,i}$ for each public key Pk_i . This optimization reduces transaction size and proof overhead while preserving security. The shared randomness is scoped strictly to a single transaction and is never reused across different transactions or for encrypting different amounts.

Security Properties. Under the elliptic-curve decisional Diffie–Hellman (DDH) assumption, ElGamal encryption provides semantic security (CPA security) for confidential amounts against any party that does not possess the corresponding secret key sk_i . An adversary cannot distinguish between encryptions of distinct plaintext amounts with probability significantly better than guessing.

Basic ElGamal encryption is malleable: an adversary can transform a ciphertext (C_1, C_2) into a different valid ciphertext encrypting a related message without knowledge of the plaintext or secret key. The Confidential MPT protocol mitigates the practical implications of this malleability through multiple, complementary mechanisms. First, all ciphertexts are covered by the transaction’s digital signature. Any unauthorized modification of (C_1, C_2) or related fields invalidates the signature and causes validators to reject the transaction. Second, all ZKPs include the exact ciphertext components (C_1, C_2) in their Fiat–Shamir challenges, together with a `TransactionContextID` that binds the proof to a specific transaction instance. Any ciphertext malleation changes the challenge and invalidates the proof. Finally, the ZKPs themselves assert semantic properties of the encrypted values (e.g., equality, consistency, or non-negativity). A malleation that alters the encrypted amount will, with overwhelming probability, violate these properties and cause proof verification to fail. Together, these measures ensure that ElGamal ciphertext malleability and off-chain chosen-ciphertext manipulation attempts are detected and rejected prior to ledger acceptance.

Reusing randomness across encryptions of the same plaintext under different public keys does not compromise semantic security in this setting. While ElGamal encryption requires fresh randomness per message for full indistinguishability, reuse of randomness for identical plaintexts does not leak additional information beyond what is already implied by plaintext equality [BBS03, Kur00]. This pattern is also used in confidential transfer systems such as Zether and Solana confidential transactions. Moreover, the protocol enforces correctness and consistency through explicit ZKPs that bind all ciphertexts to the same hidden plaintext and randomness, preventing misuse or substitution.

Public Key Registration and Rogue-Key Resistance. When shared encryption randomness is used across multiple recipients within a single transaction, the security of ElGamal encryption assumes that all recipient public keys are honestly generated and correspond to independently held secret keys. To enforce this assumption and prevent rogue-key attacks, the protocol requires that any account registering an ElGamal public key for confidential transfers provide a one-time proof of knowledge of the corresponding secret key [BBS03].

This proof is verified at key registration or initialization time and ensures that no adversary can register a maliciously related public key that could exploit shared randomness across ciphertexts. Additionally, requiring proof of knowledge prevents accidental lock-in scenarios in which a confidential public key is registered without the account holder actually knowing the corresponding secret key (e.g., due to misgeneration or misconfiguration), thereby avoiding irreversible loss of access to confidential funds.

ElGamal Decryption

This function allows the legitimate recipient of an ElGamal ciphertext to recover the original amount. Decryption is performed exclusively by the account holder (client-side) who possesses the corresponding private key and is never performed by network validators. The recipient computes $sk \cdot C_1$, subtracts it from C_2 , and obtains $mG = C_2 - (sk \cdot C_1)$. The final step is to recover the original scalar amount m from the elliptic curve point mG , which involves solving a small discrete logarithm problem (DLP). This is an intentional design choice enabling efficient balance viewing by authorized users. Section 2 presents efficient methods for solving small DLPs.

Homomorphic Operations

The ElGamal scheme used in Confidential MPTs is linearly homomorphic, allowing arithmetic operations on encrypted values without decryption. This property enables validators to update confidential balances on-ledger without revealing plaintext amounts. All homomorphic operations are implemented using elliptic curve group operations. An encryption of value m takes the form

$$\text{Enc}(m) = (C_1, C_2) = (rG, mG + rPk),$$

where r is a random scalar and Pk is the public key. Homomorphic addition combines ciphertexts $\text{Enc}(a) = (C_{1a}, C_{2a})$ and $\text{Enc}(b) = (C_{1b}, C_{2b})$ to produce

$$\text{Enc}(a + b) = (C_{1a} + C_{1b}, C_{2a} + C_{2b}).$$

Homomorphic subtraction similarly yields

$$\text{Enc}(a - b) = (C_{1a} - C_{1b}, C_{2a} - C_{2b}).$$

Scalar multiplication by a public scalar k is defined as

$$k \cdot \text{Enc}(a) = (kC_1, kC_2).$$

Zero-Knowledge Proofs (ZKPs)

This section details the specific ZKPs employed by the Confidential MPTs protocol. These proofs allow participants to verify critical properties of confidential transactions (e.g., amount correctness, range adherence, solvency) without revealing secret amounts [BS20, Sma16].

TransactionContextID (proof context binding).

All non-interactive zero-knowledge proofs (NIZKs) in this specification are bound to a context identifier, denoted `TransactionContextID`, which is included as an input to Fiat-Shamir challenge generation. This identifier is constructed exclusively from transaction fields that are known at proof-generation time and from replay-prevention state that is deterministically enforced by the XRPL. In particular, no fields that depend on ledger placement (e.g., `LedgerIndex`) or on the transaction hash itself (e.g., `TxHash`) are included, avoiding circular dependencies and nondeterminism. Concretely, for any Confidential MPT transaction, we define

$$\text{TransactionContextID} := H(\text{TxType} \parallel \text{Account} \parallel \text{MPTokenIssuanceID} \parallel \text{SequenceOrTicket} \parallel \text{TxSpecific}),$$

where `SequenceOrTicket` is the standard XRPL anti-replay identifier (either the account `Sequence` number or a `Ticket`), and `TxSpecific` contains additional fields required to bind the proof to the intended transaction semantics and, where needed, the relevant on-ledger freshness state.

Note that `TransactionContextID` is only one component of proof binding: each proof's Fiat-Shamir challenge also includes its own direct public inputs (e.g., ciphertexts, commitments, public keys, and transcript commitments). Thus, `TransactionContextID` provides transaction- and state-context binding, while the full proof transcript binds the proof to the specific statement being proved.

Send and ConvertBack binding. For transaction types whose soundness depends on a specific snapshot of the sender’s confidential spending balance—namely CONFIDENTIALMPTSEND and CONFIDENTIALMPTCONVERTBACK—we define

$$\text{TxSpecific} := \text{Receiver} \parallel \text{CB_S_Version}.$$

For CONFIDENTIALMPTSEND, *Receiver* is the destination account. For CONFIDENTIALMPTCONVERTBACK, where value is restored to the originating account in public form, *Receiver* is set equal to *Account* to preserve a uniform context structure.

The inclusion of *CB_S_Version* provides freshness binding for proofs that consume the sender’s confidential spending balance, preventing replay or double-spend attempts using proofs generated against stale spending state.

Convert binding. For CONFIDENTIALMPTCONVERT, the converted amount m is publicly revealed. The proof transcript additionally includes the group element mG , ensuring that the plaintext amount provided in the transaction fields is exactly the value used in the ElGamal ciphertext computation ($C_2 = mG + r \cdot Pk$). To maintain a uniform definition of *TransactionContextID* across all Confidential MPT transaction types, we define

$$\text{TxSpecific} := \text{Account} \parallel 0.$$

The constant 0 serves as an explicit sentinel indicating that no confidential spending-balance version is involved in this transaction type.

Clawback binding. For CONFIDENTIALMPTCLAWBACK, the transaction is initiated and authorized by the issuer rather than the token holder. To avoid allowing the holder to invalidate an issuer-prepared clawback by deliberately changing *CB_S_Version* through intervening transactions, we omit version binding and define

$$\text{TxSpecific} := \text{Holder} \parallel 0.$$

Here, *Holder* identifies the target account, while the constant 0 serves as a static domain-separation tag. Replay and substitution resistance are still preserved because the clawback proof is bound through its full transcript to the concrete public statement being proved, including the issuer-side ciphertext and the revealed plaintext amount.

B Proof of Knowledge of Secret Key (PoK) for Public Key Registration

When an account registers an ElGamal public key for confidential transfers, it must prove knowledge of the corresponding secret key. Formally, given a public key $Pk \in \mathbb{G}$, the prover demonstrates knowledge of $sk \in \mathbb{Z}_q$ such that $Pk = sk \cdot G$, where G is the fixed generator of the elliptic-curve group.

This proof serves two security objectives. First, it prevents rogue-key attacks in multi-recipient settings where shared encryption randomness may be used. Without proof of possession, an adversary could register a maliciously related public key and attempt to exploit linear relations across ciphertexts. Second, it prevents accidental key misregistration, ensuring that no confidential public key is accepted unless the registering account actually controls the corresponding secret key. The proof is instantiated as a Schnorr-style Σ -protocol rendered non-interactive via the Fiat–Shamir transform. The transcript is domain-separated and bound to the *TransactionContextID* to prevent replay or cross-context reuse. Under the discrete logarithm assumption in $\mathbb{G}$, the proof is sound and zero-knowledge in the random oracle model. Proof size is one group element and one scalar (65 bytes using compressed secp256k1 encoding).

Statement to Prove. Given a public key $Pk \in \mathbb{G}$, prove knowledge of $sk \in \mathbb{Z}_q$ such that: $Pk = sk \cdot G$. The public inputs are

- Pk : the ElGamal public key being registered;
- G : the fixed generator of the elliptic-curve group of prime order q ;
- *TransactionContextID*: transcript-binding context (must include the registration transaction type, registering account, token issuance identifier if applicable, and sequence/ticket).

Witness. $sk \in \mathbb{Z}_q$: the secret key corresponding to Pk .

Protocol (Schnorr PoK with Fiat–Shamir). Prover steps are

1. Sample $k \xleftarrow{\$} \mathbb{Z}_q$ and compute the commitment $T = k \cdot G$.
2. Compute the challenge scalar $e = H(\text{"MPT_POK_SK_REGISTER"} \parallel Pk \parallel T \parallel \text{TransactionContextID})$.
3. Compute the response $s = k + e \cdot sk \pmod{q}$.
4. Output the proof $\pi_{\text{PoK}} = (T, s)$.

Verifier.

1. Recompute the challenge $e' = H(\text{"MPT_POK_SK_REGISTER"} \parallel Pk \parallel T \parallel \text{TransactionContextID})$.
2. Accept iff $s \cdot G \stackrel{?}{=} T + e' \cdot Pk$.

Proof Size. The proof consists of one elliptic-curve point T and one scalar s . Using compressed secp256k1 encoding, this yields:

$$33 \text{ bytes} + 32 \text{ bytes} = 65 \text{ bytes}.$$

Security Properties.

- **Soundness:** Under the discrete logarithm assumption, no adversary can produce a valid proof for a public key Pk without knowing the corresponding secret key sk .
- **Zero-Knowledge:** The proof reveals no information about sk beyond the fact that it is known.
- **Non-Interactivity:** The proof is made non-interactive via the Fiat–Shamir transform in the random oracle model.
- **Context Binding:** Inclusion of `TransactionContextID` binds the proof to a specific registration transaction instance and prevents replay or reuse in a different context.

C Optimized Ciphertext–Amount Consistency Protocol

Certain transaction types (e.g., `CONFIDENTIALMPTCONVERT`, `CONFIDENTIALMPTCONVERTBACK`, and `CONFIDENTIALMPTCLAWBACK`) require demonstrating that an EC–ElGamal ciphertext correctly encrypts a publicly declared plaintext amount m . Let (C_1, C_2) be an EC–ElGamal ciphertext under public key $Pk = sk \cdot G$, defined as

$$C_1 = rG, \quad C_2 = mG + rPk,$$

for some randomness $r \in \mathbb{Z}_q$. The prover must demonstrate that the ciphertext encodes the claimed plaintext m .

Deterministic Verification. When the encryption randomness r is disclosed (as in conversion-style transactions), validators verify correctness by checking the group equalities:

$$C_1 \stackrel{?}{=} rG, \quad C_2 - mG \stackrel{?}{=} rPk.$$

No zero-knowledge proof is required in this case.

Zero-Knowledge Variant. When r is not revealed (as in the clawback transaction), ciphertext–plaintext consistency is established using a Chaum–Pedersen proof of equality of discrete logarithms, demonstrating knowledge of r such that

$$C_1 = rG \quad \text{and} \quad C_2 - mG = rPk.$$

The proof is rendered non-interactive via Fiat–Shamir and bound to the `TransactionContextID`. A detailed specification is provided in Appendix C. Under the discrete logarithm assumption, the proof guarantees soundness and zero-knowledge in the random oracle model. Proof size is two group elements and one scalar (approximately 98 bytes using compressed secp256k1 encoding). We describe the corresponding NIZKP formulation for Chaum–Pedersen Proof of Knowledge of r . The prover demonstrates knowledge of r such that: $C_1 = r \cdot G$ and $C_2 - m \cdot G = r \cdot Pk$.

Public Inputs. (C_1, C_2) , recipient public key Pk , plaintext amount m , and transaction context identifier.

Witness. The encryption randomness $r \in \mathbb{Z}_q$.

Protocol.

1. Sample $t \xleftarrow{\$} \mathbb{Z}_q$ and compute $T_1 = t \cdot G$, $T_2 = t \cdot Pk$.
2. Compute

$$e = H(\text{"MPT_POK_PLAINTEXT_PROOF"} \parallel C_1 \parallel C_2 \parallel Pk \parallel mG \parallel T_1 \parallel T_2 \parallel \text{TransactionContextID}).$$

3. Compute $s = t + e \cdot r \pmod{q}$.

Verification. Accept if:

$$\begin{aligned} s \cdot G &= T_1 + e \cdot C_1, \\ s \cdot Pk &= T_2 + e \cdot (C_2 - m \cdot G). \end{aligned}$$

Remark for Clawback via Issuer Secret Key. In a `ConfidentialMPTClawback` transaction, the issuer does not know the original encryption randomness r . Instead, the same equality proof is instantiated by exchanging the roles of bases and witness:

$$Pk_{\text{iss}} = sk_{\text{iss}} \cdot G \quad \text{and} \quad C_2 - m \cdot G = sk_{\text{iss}} \cdot C_1.$$

Here, the witness is the issuer secret key sk_{iss} , and the proof demonstrates issuer authority rather than knowledge of r . The resulting protocol is a standard Chaum–Pedersen NIZKP with identical structure and verification equations.

Proof Size and Efficiency. The Chaum–Pedersen proof consists of two curve points and one scalar (98 bytes using compressed secp256k1 encoding). No proof data is included for deterministic verification transactions.

Security Properties.

- **Soundness:** Valid proofs imply ciphertext–plaintext consistency.
- **Zero-Knowledge:** No information about r or sk_{iss} is revealed.
- **Authority:** Only the issuer can authorize clawback.
- **Efficiency:** ZKPs are avoided when deterministic checks suffice.
- **Context Binding:** `TransactionContextID` prevents replay and cross-transaction substitution.

D Proof of Equality of Plaintexts with Shared Randomness

In CONFIDENTIALMPTSEND, the same transfer amount must be encrypted under multiple public keys (e.g., sender, receiver, issuer mirror, and optional auditor key). To ensure consistency across all ciphertexts, the protocol proves that they encrypt the same secret amount m using a common encryption randomness r . Let $Pk_1, \dots, Pk_N$ be public keys. For shared randomness $r \in \mathbb{Z}_q$ and amount $m \in \mathbb{Z}_q$, the ciphertexts are defined as:

$$C_1 = rG, \quad C_{2,i} = mG + rPk_i \quad \text{for } i = 1, \dots, N.$$

The prover demonstrates knowledge of (m, r) such that the above relations hold simultaneously for all recipients. The proof is instantiated as a multi-recipient Schnorr-style Σ -protocol proving equality of discrete logarithms across multiple bases. It is rendered non-interactive via the Fiat-Shamir transform and transcript-bound to the TransactionContextID.

Under the discrete logarithm assumption, the proof is sound and zero-knowledge in the random oracle model. It guarantees that all ciphertexts encode the identical plaintext amount, preventing balance divergence between participants. The proof consists of $(1 + N)$ group elements and two scalars. For the typical case $N = 3$ or $N = 4$, the size is approximately 200–230 bytes using compressed secp256k1 encoding. **Ciphertext Definitions.** Let N be the number of recipients. For a secret amount $m \in \mathbb{Z}_q$ and shared randomness $r \in \mathbb{Z}_q$, the ciphertexts are defined as:

$$C_1 = r \cdot G, \\ C_{2,i} = m \cdot G + r \cdot Pk_i \quad \text{for } i = 1, \dots, N.$$

Statement to Prove. The prover demonstrates knowledge of (m, r) such that the above relations hold for all $i \in \{1, \dots, N\}$. This establishes that all ciphertexts encrypt the same plaintext amount using the same randomness, without revealing either m or r .

Protocol (Multi-Recipient Schnorr-style Proof).

Public Inputs.

- Shared component C_1 and ciphertext components $C_{2,1}, \dots, C_{2,N}$;
- recipient public keys $Pk_1, \dots, Pk_N$;
- transaction context identifier TransactionContextID.

Witness. The prover's witness consists of the secret amount m and the shared encryption randomness r .

Prover Steps.

1. Sample random blinding scalars $k_m, k_r \xleftarrow{\$} \mathbb{Z}_q$.
2. Compute commitments:

$$T_r = k_r \cdot G, \\ T_{m,i} = k_m \cdot G + k_r \cdot Pk_i \quad \text{for } i = 1, \dots, N.$$

3. Compute the challenge

$$e = H(\text{"MPT_POK_SAME_PLAINTEXT_SHARED_R"} \parallel C_1 \parallel \{C_{2,i}, Pk_i\}_{i=1}^N \parallel T_r \parallel \{T_{m,i}\}_{i=1}^N \parallel \text{TransactionContextID}).$$

4. Compute responses:

$$s_m = k_m + e \cdot m \pmod{q}, \\ s_r = k_r + e \cdot r \pmod{q}.$$

Verification. The verifier recomputes the challenge e' and accepts the proof $(T_r, \{T_{m,i}\}_{i=1}^N, s_m, s_r)$ if:

$$s_r \cdot G \stackrel{?}{=} T_r + e' \cdot C_1, \\ s_m \cdot G + s_r \cdot Pk_i \stackrel{?}{=} T_{m,i} + e' \cdot C_{2,i} \quad \text{for all } i = 1, \dots, N.$$

Proof Size and Efficiency. The proof consists of one group element T_r , N group elements $\{T_{m,i}\}$, and two scalars (s_m, s_r) . Using compressed secp256k1 encoding, this yields:

- **Points:** $(1 + N) \times 33$ bytes;
- **Scalars:** 2×32 bytes.

For $N = 4$ (sender, receiver, issuer), the total proof size is $5 \cdot 33 + 2 \cdot 32 = 229$ bytes.

Security Properties.

- **Soundness:** A valid proof guarantees that all ciphertexts encrypt the same plaintext amount using shared randomness.
- **Zero-Knowledge:** The proof reveals no information about m or r beyond correctness.
- **Consistency:** The construction ensures that sender, receiver, and issuer balances are updated by identical amounts.
- **Efficiency:** Shared randomness significantly reduces proof size and verification cost.

E Linking ElGamal Ciphertexts and Pedersen Commitments

In CONFIDENTIALMPTSEND and CONFIDENTIALMPTCONVERTBACK, the protocol employs two distinct cryptographic representations of confidential values: (i) EC-ElGamal ciphertexts, which enable homomorphic balance updates on-ledger, and (ii) Pedersen commitments, which are required as inputs to Bulletproof range proofs. Because Bulletproofs require commitment generators that are independent of the ElGamal encryption generators, a value encrypted under ElGamal is not implicitly linked to a value committed in a Pedersen commitment.

To ensure correctness, the protocol includes explicit ZKPs that establish consistency between ElGamal ciphertexts and Pedersen commitments. These linkage proofs prevent a malicious prover from presenting a valid range proof for one value while updating balances using ciphertexts encoding a different value.

Let (C_1, C_2) be an EC-ElGamal ciphertext under public key Pk , and let PC be a Pedersen commitment. The prover must demonstrate that both objects encode the same value $v \in \mathbb{Z}_q$. Depending on transaction context, the prover may know either:

- the encryption randomness r (for freshly generated ciphertexts), or
- the secret key sk corresponding to Pk (for existing balance ciphertexts).

Accordingly, the protocol instantiates one of two Schnorr-style Σ -protocols proving equality of discrete logarithms across independent bases. Both variants are rendered non-interactive via Fiat-Shamir and transcript-bound to the TransactionContextID. Depending on the transaction context, the prover may know either the ElGamal *encryption randomness* or the corresponding *secret key*. Accordingly, the protocol defines two Schnorr-style linkage proofs.

Variant A: Encryption-Randomness-Based Linkage (Amount Commitment). This variant is used for freshly created ciphertexts, such as the transfer amount m in CONFIDENTIALMPTSEND, where the prover knows the ElGamal encryption randomness.

Statement. Let $\text{Enc}_{pk}(m) = (C_1, C_2) = (rG, mG + rpk)$ be an EC-ElGamal encryption of amount m under public key pk , and let $PC_m = mG + \rho H$ be a Pedersen commitment to the same amount m . The prover demonstrates knowledge of (m, r, ρ) such that:

$$\begin{aligned} C_1 &= rG, \\ C_2 &= mG + rpk, \\ PC_m &= mG + \rho H. \end{aligned}$$

Public Inputs.

- (C_1, C_2) : ElGamal ciphertext,
- pk : ElGamal public key,
- PC_m : Pedersen commitment,
- $\text{TransactionContextID}$.

Witness. (m, r, ρ) .

Prover. The prover samples random scalars $k_m, k_r, k_\rho \xleftarrow{\$} \mathbb{Z}_q$ and computes:

$$\begin{aligned} T_1 &= k_r G, \\ T_2 &= k_m G + k_r \text{pk}, \\ T_3 &= k_m G + k_\rho H. \end{aligned}$$

The challenge is derived via Fiat–Shamir:

$$e = H(\text{"MPT_ELGAMAL_PEDERSEN_LINK_R"} \parallel C_1 \parallel C_2 \parallel \text{pk} \parallel PC_m \parallel T_1 \parallel T_2 \parallel T_3 \parallel \text{TransactionContextID}).$$

Responses:

$$\begin{aligned} s_m &= k_m + em \pmod{q}, \\ s_r &= k_r + er \pmod{q}, \\ s_\rho &= k_\rho + e\rho \pmod{q}. \end{aligned}$$

Verifier. The verifier recomputes e and checks:

$$\begin{aligned} s_r G &= T_1 + eC_1, \\ s_m G + s_r \text{pk} &= T_2 + eC_2, \\ s_m G + s_\rho H &= T_3 + ePC_m. \end{aligned}$$

Variant B: Secret-Key–Based Linkage (Balance Commitment). This variant is used when linking a Pedersen commitment to an *existing* ElGamal ciphertext for which the prover does not know the encryption randomness, but *does* know the corresponding secret key. This occurs when proving balance sufficiency in CONFIDENTIALMPT-SEND and CONFIDENTIALMPTCONVERTBACK.

Statement. Let $\text{pk} = sG$ be an ElGamal public key with secret key s , let $\text{Enc}_{\text{pk}}(b) = (C_1, C_2)$ be an ElGamal ciphertext encoding the confidential balance b , and let $PC_b = bG + \rho H$ be a Pedersen commitment to the same value. The prover demonstrates knowledge of (b, s, ρ) such that:

$$\begin{aligned} \text{pk} &= sG, \\ C_2 &= bG + sC_1, \\ PC_b &= bG + \rho H. \end{aligned}$$

Public Inputs.

- (C_1, C_2) : ElGamal ciphertext,
- pk : ElGamal public key,
- PC_b : Pedersen commitment,
- $\text{TransactionContextID}$.

Witness. (b, s, ρ) .

Prover. The prover samples random scalars $k_b, k_s, k_\rho \xleftarrow{\$} \mathbb{Z}_q$ and computes:

$$\begin{aligned} T_1 &= k_s G, \\ T_2 &= k_b G + k_s C_1, \\ T_3 &= k_b G + k_\rho H. \end{aligned}$$

Challenge: $e = H(\text{"MPT_ELGAMAL_PEDERSEN_LINK_SK"} \parallel C_1 \parallel C_2 \parallel \text{pk} \parallel PC_b \parallel T_1 \parallel T_2 \parallel T_3 \parallel \text{TransactionContextID})$.

Responses:

$$\begin{aligned} s_b &= k_b + eb \pmod{q}, \\ s_s &= k_s + es \pmod{q}, \\ s_\rho &= k_\rho + e\rho \pmod{q}. \end{aligned}$$

Verifier. The verifier recomputes e and checks:

$$\begin{aligned} s_s G &= T_1 + e \text{pk}, \\ s_b G + s_s C_1 &= T_2 + e C_2, \\ s_b G + s_\rho H &= T_3 + e PC_b. \end{aligned}$$

Proof Size. Each variant consists of three elliptic-curve points and three scalars. Using compressed secp256k1 encoding, the proof size is:

$$3 \times 33 \text{ bytes} + 3 \times 32 \text{ bytes} = 195 \text{ bytes}.$$

Security Properties.

- **Soundness:** A prover cannot construct a valid proof unless the ElGamal ciphertext and Pedersen commitment encode the same plaintext value.
- **Zero-Knowledge:** The proof reveals no information about m , r , or ρ beyond correctness.
- **Non-Interactivity:** The proof is made non-interactive via the Fiat–Shamir transform.
- **Context Binding:** Inclusion of TransactionContextID binds the proof to a specific transaction and balance state, preventing replay or substitution.

F Range Proof (using Bulletproofs)

The Bulletproofs protocol [BBB⁺18] enables a prover to demonstrate that a secret amount m , which is committed in a Pedersen commitment $PC_m = mG + r_m H$, lies within a specified range $[0, 2^N)$ without revealing m or its blinding factor r_m . This is crucial for enforcing non-negativity and boundedness of confidential amounts, preventing overflows, underflows, or negative values within the MPT protocol. Let the secret amount be $m \in [0, 2^N)$, and its Pedersen commitment be $PC_m = mG + r_m H$, where G, H are distinct, non-related generators of the elliptic curve group (e.g., secp256k1), $r_m \in \mathbb{Z}_q$ is the secret blinding scalar (nonce) for the commitment, $m \in \mathbb{Z}_q$ is the secret `uint64_t` plaintext scalar value (amount), and N is the bit-length of the range (e.g., 64). The prover wants to convince the verifier that $m \in [0, 2^N)$ for the secret m committed in PC_m .

Under the discrete logarithm assumption and the security of the Fiat–Shamir transform in the random oracle model, Bulletproofs provide soundness and zero-knowledge. Successful verification guarantees that the committed value lies within the specified range and prevents overflow, underflow, or negative-value creation in confidential balances. For $N = 64$ and $M = 2$, the aggregated proof consists of 18 group elements and 5 scalars, resulting in approximately 750 bytes using compressed secp256k1 encoding. Verification requires $O(\log(MN))$ multi-scalar multiplications in the elliptic-curve group.

Inputs for Proving

- PC_m : The Pedersen commitment $mG + r_mH$ to the secret value.
- m : The secret amount.
- r_m : The secret blinding scalar used in PC_m .
- N : The bit-length of the range (e.g., 64).

Inputs for Verifying

- **proof**: The byte array of the Bulletproof.
- **proof_len**: Length of the proof.
- PC_m : The Pedersen commitment $mG + r_mH$.
- N : The bit-length of the range (e.g., 64).

Formal Protocol (Bulletproof Single-Value Range Proof)

This section provides a concrete, specification-level description of the Bulletproofs single-value range proof protocol [BBB⁺18]. The notation used in this section follows the definitions in Section 2.4 of [BBB⁺18]. The prover demonstrates knowledge of scalars $(m, r_m) \in \mathbb{Z}_q^2$ such that

$$PC_m = mG + r_mH \quad \text{and} \quad 0 \leq m < 2^N,$$

without revealing m or r_m . All group operations are performed in an elliptic curve group $\mathbb{G}$ of prime order q (e.g., secp256k1), and all scalar arithmetic is performed modulo q . The protocol assumes fixed public generators:

$$G, H \in \mathbb{G}, \quad \mathbf{G} = (G_1, \dots, G_N), \quad \mathbf{H} = (H_1, \dots, H_N),$$

where all generators are distinct and deterministically derived (e.g., via hash-to-curve with domain separation). The range bit-length N is fixed by the protocol (e.g., $N = 64$). The prover writes the secret amount in binary:

$$m = \sum_{i=0}^{N-1} a_{L,i} 2^i, \quad a_{L,i} \in \{0, 1\}.$$

Define vectors $\mathbf{a}_L = (a_{L,0}, \dots, a_{L,N-1})$, $\mathbf{a}_R = \mathbf{a}_L - \mathbf{1}$. The Booleanity constraint $a_{L,i} \in \{0, 1\}$ is enforced by requiring

$$a_{L,i} \cdot a_{R,i} = 0 \quad \text{for all } i.$$

The prover samples random blinding values

$$\mathbf{s}_L, \mathbf{s}_R \leftarrow \mathbb{Z}_q^N, \quad \alpha, \rho \leftarrow \mathbb{Z}_q,$$

and computes vector commitments

$$\begin{aligned} A &= \alpha H + \langle \mathbf{a}_L, \mathbf{G} \rangle + \langle \mathbf{a}_R, \mathbf{H} \rangle, \\ S &= \rho H + \langle \mathbf{s}_L, \mathbf{G} \rangle + \langle \mathbf{s}_R, \mathbf{H} \rangle. \end{aligned}$$

Fiat–Shamir Challenges. The Bulletproof range proof is rendered non-interactive using the Fiat–Shamir transform, following the construction in Section 4.4 of [BBB⁺18]. The interactive protocol consists solely of public-coin challenges sampled uniformly from $\mathbb{Z}_q$, and these challenges are deterministically derived by hashing an incrementally constructed transcript in the random oracle model. The Fiat–Shamir transcript is constructed incrementally as the protocol progresses. Let $\mathcal{T}_0$ denote the initial transcript, which includes:

- a domain-separation tag identifying the proof type (e.g., "MPT_BULLETPROOF_RANGE");
- the fixed Bulletproof generators $(G, H, \mathbf{G}, \mathbf{H})$;
- the statement being proven, namely the Pedersen commitment PC_m and the range parameter N ;

- public protocol parameters (e.g., bit length N , aggregation count);
- the `TransactionContextID`, binding the proof to a specific transaction and ledger state.

The prover first generates the Bulletproof commitments A and S and appends them to the transcript $\mathcal{T}_1 = \mathcal{T}_0 \parallel A \parallel S$. Here, $\mathcal{T}_0$ corresponds to the statement st in the notation of [BBB⁺18]. The verifier challenges are then derived as:

$$y = H_{\text{FS}}(\mathcal{T}_1), \quad z = H_{\text{FS}}(\mathcal{T}_1 \parallel y),$$

with all hash outputs interpreted as elements of $\mathbb{Z}_q$.

Polynomial Construction. Define vector-valued polynomials:

$$\mathbf{l}(X) = \mathbf{a}_L - z\mathbf{1} + \mathbf{s}_L X,$$

$$\mathbf{r}(X) = (\mathbf{a}_R + z\mathbf{1} + \mathbf{s}_R X) \circ \mathbf{y} + z^2 \mathbf{2}^N,$$

where $\mathbf{y} = (1, y, y^2, \dots, y^{N-1})$, $\mathbf{2}^N = (2^0, \dots, 2^{N-1})$, and $\circ$ denotes element-wise multiplication. Define the scalar polynomial

$$t(X) = \langle \mathbf{l}(X), \mathbf{r}(X) \rangle = t_0 + t_1 X + t_2 X^2.$$

Polynomial Commitments. The prover samples $\tau_1, \tau_2 \leftarrow \mathbb{Z}_q$ and computes

$$T_1 = t_1 G + \tau_1 H, \quad T_2 = t_2 G + \tau_2 H.$$

After computing the polynomial commitments T_1 and T_2 , the transcript is extended as $\mathcal{T}_2 = \mathcal{T}_1 \parallel y \parallel z \parallel T_1 \parallel T_2$, and the challenge $x = H_{\text{FS}}(\mathcal{T}_2)$ is derived.

Evaluation and Responses. The prover evaluates

$$\mathbf{l} = \mathbf{l}(x), \quad \mathbf{r} = \mathbf{r}(x), \quad t = t(x),$$

and computes response scalars

$$\tau_x = \tau_2 x^2 + \tau_1 x + z^2 r_m, \quad \mu = \alpha + \rho x.$$

Inner Product Argument. The final step of the Bulletproof range proof reduces verification of the polynomial identity to a proof of a single inner-product relation using the Inner Product Argument (IPA) of [BBB⁺18]. The prover demonstrates knowledge of vectors $\mathbf{l}, \mathbf{r} \in \mathbb{Z}_q^N$ such that

$$t = \langle \mathbf{l}, \mathbf{r} \rangle = \sum_{i=1}^N l_i r_i,$$

where $\mathbf{l} = \mathbf{l}(x)$ and $\mathbf{r} = \mathbf{r}(x)$ are the evaluations of the previously defined vector polynomials at the Fiat–Shamir challenge point x . Prior to the IPA, the verifier implicitly checks that the prover’s commitments define a group element

$$P = \langle \mathbf{l}, \mathbf{G}' \rangle + \langle \mathbf{r}, \mathbf{H}' \rangle + t \cdot U,$$

where:

- $\mathbf{G}', \mathbf{H}'$ are generator vectors derived deterministically from $(\mathbf{G}, \mathbf{H})$ and the challenge y ,
- $U \in \mathbb{G}$ is a fixed generator used to encode the inner-product value,
- all generators are public and fixed by the protocol.

Recursive Reduction. The IPA proceeds recursively by halving the dimension of the vectors at each round. For $N = 2^k$, the prover performs $k = \log_2 N$ rounds. In each round, the prover:

- splits $\mathbf{l}$ and $\mathbf{r}$ into left and right halves;
- computes two commitment points $L_i, R_i \in \mathbb{G}$ encoding cross inner-products between the halves;

- appends (L_i, R_i) to the transcript, from which the verifier derives a Fiat–Shamir challenge scalar $u_i \in \mathbb{Z}_q$ as follows: The IPA itself proceeds over multiple rounds; in each round i , the prover produces points (L_i, R_i) , which are appended to the transcript sequentially as $\mathcal{T}_{i+1} = \mathcal{T}_i \parallel L_i \parallel R_i$, and the corresponding challenge $u_i = H_{\text{FS}}(\mathcal{T}_{i+1})$ is derived. This process continues until the inner-product argument terminates.

Using the challenge u_i , both prover and verifier fold the vectors and generators as:

$$\mathbf{l} \leftarrow u_i \mathbf{l}_L + u_i^{-1} \mathbf{l}_R, \quad \mathbf{r} \leftarrow u_i^{-1} \mathbf{r}_L + u_i \mathbf{r}_R,$$

and similarly update $(\mathbf{G}', \mathbf{H}', P)$, reducing the problem size by a factor of two. After $\log_2 N$ rounds, the vectors reduce to single scalars (a, b) . The prover reveals (a, b) .

Soundness and Zero-Knowledge. The IPA is a proof of knowledge that is:

- *Sound*, assuming the hardness of the discrete logarithm problem in $\mathbb{G}$ and the security of the Fiat–Shamir transform;
- *Zero-knowledge*, as the transcript can be simulated without knowledge of $(\mathbf{l}, \mathbf{r})$ beyond the value of t .

Proof Size and Complexity. For vectors of length N , the IPA contributes $2 \log_2 N$ group elements (L_i, R_i) and two scalars (a, b) to the proof. For $N = 64$, this corresponds to 12 elliptic curve points from the IPA component of the Bulletproof.

Final Proof The Bulletproof range proof is the tuple $(A, S, T_1, T_2, \tau_x, \mu, t, (L_1, R_1), \dots, (L_k, R_k), a, b)$, which for $N = 64$ results in a proof size of approximately 700–800 bytes.

Verification. Given the proof $\pi = (A, S, T_1, T_2, \tau_x, \mu, t, (L_1, R_1), \dots, (L_k, R_k), a, b)$, the Pedersen commitment PC_m , the range parameter N , and the `TransactionContextID`, the verifier proceeds as follows. The verifier reconstructs the Fiat–Shamir transcript and deterministically derives all challenges:

$$y = H_{\text{FS}}(\mathcal{T}_0 \parallel A \parallel S), \quad z = H_{\text{FS}}(\mathcal{T}_0 \parallel A \parallel S \parallel y),$$

where $\mathcal{T}_0$ includes the domain-separation tag, generators $(G, H, \mathbf{G}, \mathbf{H})$, the statement (PC_m, N) , and the `TransactionContextID`.

After appending T_1 and T_2 , the verifier derives: $x = H_{\text{FS}}(\mathcal{T}_1 \parallel T_1 \parallel T_2)$, where $\mathcal{T}_1 = \mathcal{T}_0 \parallel A \parallel S \parallel y \parallel z$. The verifier then computes the scalar

$$\delta(y, z) = (z - z^2) \langle \mathbf{1}, \mathbf{y} \rangle - z^3 \langle \mathbf{1}, \mathbf{2}^N \rangle,$$

where $\mathbf{y} = (1, y, y^2, \dots, y^{N-1})$ and $\mathbf{2}^N = (2^0, \dots, 2^{N-1})$. The verifier then checks the polynomial commitment identity:

$$tG + \tau_x H \stackrel{?}{=} z^2 PC_m + \delta(y, z)G + xT_1 + x^2 T_2.$$

Failure of this check causes immediate rejection.

Inner Product Argument Verification. The verifier constructs the initial commitment

$$P = A + xS - z \langle \mathbf{1}, \mathbf{G} \rangle + (z\mathbf{y} + z^2 \mathbf{2}^N) \cdot \mathbf{H} - \mu H + tU,$$

where U is a fixed generator used for the inner-product encoding. For each round $i = 1, \dots, k$, the verifier:

- derives a challenge $u_i = H_{\text{FS}}(\mathcal{T}_i \parallel L_i \parallel R_i)$;
- updates the commitment $P \leftarrow u_i^2 L_i + P + u_i^{-2} R_i$;
- folds the generator vectors accordingly.

After $\log_2 N$ rounds, the verifier checks the final scalar equation:

$$P \stackrel{?}{=} aG^* + bH^* + (ab)U,$$

where G^* and H^* are the folded generators resulting from the recursive reductions.

Acceptance. The proof is accepted if and only if all of the above checks succeed. Successful verification implies that the committed value m satisfies $0 \leq m < 2^N$ and that PC_m is a valid Pedersen commitment to m .

Security Properties.

- **Soundness:** It is computationally infeasible for a malicious prover to construct a valid Bulletproof for a false statement (e.g., proving an amount m is in range when m is negative or exceeds $2^N - 1$). This relies on the hardness of the DLP problems on secp256k1 and the security of the Fiat–Shamir transform.
- **Zero-Knowledge:** The proof reveals no information about the committed secret value m or its blinding factor r_m beyond the truth of the proven statement.
- **Non-Interactive:** Proofs are non-interactive, making them well-suited for blockchain environments.
- **No Trusted Setup:** Bulletproofs do not require a trusted setup phase, eliminating a potential single point of failure.
- **Binding:** The inclusion of TransactionContextID in challenge generation binds the proof to a specific transaction, preventing replay attacks or proof substitution.

Aggregated Range Proof.

The Confidential MPT protocol utilizes aggregated Bulletproofs [BBB⁺18] to demonstrate that multiple secret values committed in Pedersen commitments lie within the range $[0, 2^N)$ simultaneously. This is used in the ConfidentialMPTSend transaction to prove both the transfer amount m and the post-transfer remainder balance $b - m$ are valid 64-bit values, thereby enforcing non-negativity and sender solvency in a single proof. Let $M = 2$ be the number of values being proven and $N = 64$ be the bit-length. The prover demonstrates knowledge of M pairs of scalars (m_j, ρ_j) for $j \in \{1, \dots, M\}$ such that:

$$PC_j = m_j G + \rho_j H \quad \text{and} \quad 0 \leq m_j < 2^N$$

where PC_1 corresponds to the transfer amount PC_m and PC_2 corresponds to the internally derived remainder commitment $PC_{rem} := PC_b - PC_m$.

Aggregated Construction. The protocol extends the single-value range proof by concatenating the bit-vectors for all M values into combined vectors of length MN .

- **Vector Commitments:** The commitments A and S are computed over the concatenated bit-vectors $\mathbf{a}_L, \mathbf{a}_R, \mathbf{s}_L, \mathbf{s}_R \in \mathbb{Z}_q^{MN}$ and generator vectors $\mathbf{G}, \mathbf{H} \in \mathbb{G}^{MN}$.
- **Aggregated Polynomials:** The vector polynomials $\mathbf{l}(X)$ and $\mathbf{r}(X)$ are defined over MN dimensions, and the scalar polynomial $t(X) = \langle \mathbf{l}(X), \mathbf{r}(X) \rangle$ encodes the constraints for all committed values.
- **Verification Identity:** The verifier re-derives challenges y, z, x and checks the aggregated polynomial identity:

$$tG + \tau_x H \stackrel{?}{=} \sum_{j=1}^M z^{2j} \cdot PC_j + \delta(y, z)G + xT_1 + x^2T_2$$

where $\delta(y, z)$ is a constant scalar that accounts for the combined powers of y and z across all M values.

Complexity and Size. Aggregation allows the proof size to grow logarithmically with the number of values M .

- **Proof Size:** An aggregated range proof for M values of N bits consists of $2\lceil \log_2(MN) \rceil + 4$ group elements and 5 scalars.
- **Efficiency:** For $N = 64$ and $M = 2$, the proof contains 18 group elements and 5 scalars, resulting in a total size of approximately 754 bytes using compressed secp256k1 points.
- **Bandwidth Savings:** This construction is substantially smaller than two independent range proofs, reducing transaction size by nearly an order of magnitude compared to bit-decomposition methods.

G Transaction Field Definitions

Field	Description	Requirement
TransactionType	Identifies the transaction as ConfidentialMPTConvert .	Mandatory
Account	The XRPL account initiating the conversion.	Mandatory
MPTokenIssuanceID	The unique identifier of the associated MPT issuance.	Mandatory
MPTAmount	Public plaintext amount m to convert from public to confidential form.	Mandatory
HolderEncryptionPublicKey	The holder's encryption public key pk_A . Required only when initializing a confidential balance and no key is already registered; forbidden otherwise.	Conditional
HolderEncryptedAmount	A 66-byte ElGamal ciphertext credited to the holder's confidential inbox balance CB_{IN} .	Mandatory
IssuerEncryptedAmount	A 66-byte ElGamal ciphertext credited to the issuer's encrypted mirror balance.	Mandatory
AuditorEncryptedAmount	A 66-byte ElGamal ciphertext for auditor visibility. Required when an auditor key is configured on the issuance.	Conditional
BlindingFactor	The 32-byte scalar used during ElGamal encryption. Validators use it to verify ciphertext consistency with MPTAmount .	Mandatory
ZKProof	A Schnorr proof of knowledge proving ownership of the secret key corresponding to HolderElGamalPublicKey . Required only when the public key is included.	Conditional

Table 3: Fields of the **ConfidentialMPTConvert** transaction.

Field	Description	Requirement
TransactionType	Identifies the transaction as ConfidentialMPTSend .	Mandatory
Account	The sender's XRPL account.	Mandatory
Destination	The receiver's XRPL account.	Mandatory
MPTokenIssuanceID	Identifier of the MPT issuance being transferred.	Mandatory
SenderEncryptedAmount	Ciphertext used to homomorphically debit the sender's confidential spending balance.	Mandatory
DestinationEncryptedAmount	Ciphertext credited to the receiver's confidential inbox balance.	Mandatory
IssuerEncryptedAmount	Ciphertext used to update the issuer mirror balance.	Mandatory
ZKProof	Zero-knowledge proof bundle establishing equality, linkage, and range sufficiency.	Mandatory
BalanceCommitment	A cryptographic commitment to the sender's confidential spending balance.	Mandatory
AmountCommitment	A cryptographic commitment to the transfer amount.	Mandatory
AuditorEncryptedAmount	Ciphertext for auditor visibility. Required when an auditor key is configured on the issuance.	Conditional

Table 4: Fields of the **ConfidentialMPTSend** transaction.

Field	Description	Requirement
TransactionType	The transaction ConfidentialMPTConvertBack .	Mandatory
Account	The XRPL account performing the confidential-to-public conversion.	Mandatory
MPTokenIssuanceID	The unique identifier of the associated MPT issuance.	Mandatory
MPTAmount	Plaintext amount credited back to the public MPT balance.	Mandatory
HolderEncryptedAmount	A 66-byte ciphertext subtracted from the holder's confidential spending balance.	Mandatory
IssuerEncryptedAmount	A 66-byte ciphertext subtracted from the issuer's encrypted mirror balance.	Mandatory
AuditorEncryptedAmount	A 66-byte ciphertext for auditor visibility. Required when an auditor key is configured on the issuance.	Conditional
BlindingFactor	The 32-byte scalar used during ElGamal encryption. Validators use it to verify ciphertext consistency with MPTAmount .	Mandatory
BalanceCommitment	A 33-byte cryptographic commitment to the holder's confidential spending balance.	Mandatory
ZKProof	A proof bundle containing the Pedersen linkage proof and the range proof, establishing correct balance linkage and non-negative remainder.	Mandatory

Table 5: Fields of the **ConfidentialMPTConvertBack** transaction.

Field	Description	Requirement
TransactionType	Identifies the transaction as ConfidentialMPTClawback .	Mandatory
Account	The issuer account initiating the clawback transaction.	Mandatory
Holder	The holder account from which confidential funds are being clawed back.	Mandatory
MPTokenIssuanceID	The unique identifier of the associated MPT issuance.	Mandatory
MPTAmount	The plaintext total amount removed from the holder's confidential balance and credited back according to issuance clawback rules.	Mandatory
ZKProof	An equality proof validating that the revealed plaintext amount matches the corresponding issuer-encrypted balance representation.	Mandatory

Table 6: Fields of the **ConfidentialMPTClawback** transaction.